Prague University of Economics and Business
Faculty of Informatics and Statistics

A Data Integration Framework for Enterprise Application Security

MASTER'S THESIS

Study program: Applied Data Analytics and Artificial Intelligence
Specialization: Retail and E-Commerce Data Analytics

Author: Bc. Vojtěch Kraus
Supervisor: Ing. Aleš Kotuč

Prague, May 2026

Declaration on the Use of Artificial Intelligence

I confirm that artificial intelligence tools were used in the preparation of the submitted work. Specifically, they were used in the following ways:

- improvement of grammar, language style, and conciseness of the thesis, including review cycles that shortened the text by rephrasing longer passages and relocating details into the attached product documentation,
- generation of source code for figures in the thesis,
- generation of source code attached to the thesis (`mvp.zip`),
- generation of product documentation attached to the thesis (`docs.zip`),
- assistance with the LaTeX build scripts.

Each use of any of the above methods is separately documented in an appendix to the thesis, which provides a more detailed explanation of which tool was used in which specific part of the work; the author's own contribution is highlighted.

I confirm that:

- my work is original and was prepared in accordance with ethical standards, in particular the Code of Ethics of the Prague University of Economics and Business; that all information sources used are properly cited in the work; and that any generated content has been verified by me;
- I accept full responsibility for the entire content of the thesis.

Acknowledgements

I would like to thank Ing. Aleš Kotuč for his guidance and valuable feedback.

Abstract

To protect their business-critical applications from cybersecurity threats, enterprise security teams rely on numerous tools to detect and resolve vulnerabilities in application source code, configuration, dependencies, CI/CD pipelines, and runtime infrastructure. These tools use different integration patterns, protocols, and data formats, making it difficult to consistently parse, deduplicate, triage, and group their findings. In large environments, additional challenges emerge when linking all findings to the corresponding business applications. This thesis presents a data framework to tackle those challenges, delivering three distinct contributions: (1) a requirements specification for an application security data platform, (2) a reusable and extensible design covering architecture, data model, and medallion-based data pipeline, and (3) a reference implementation built on the Databricks platform. Together, these contributions offer application security teams a foundation for building a robust data platform independent of individual cybersecurity vendors, relieving them of risks usually associated with vendor lock-in.

Keywords

application security, software security, DevSecOps, security integration, data consolidation, medallion architecture, Databricks

Contents

List of Figures

List of Tables

List of source codes

Introduction

Motivation

To protect business critical applications, security teams rely on a large variety of tools: Static Application Security Testing (SAST) scanners analyze source code, Software Composition Analysis (SCA) tools check software dependencies, Dynamic Application Security Testing (DAST) probes running applications, and secret scanners flag exposed credentials in code (Chess & West, 2007; Howard & LeBlanc, 2006; Stuttard & Pinto, 2011).

Further detection runs against build and deployment pipelines, which face their own classes of threats such as malicious dependencies published to public package registries, compromised build agents injecting code into release artifacts, tampered container base images, or leaked Continuous Integration and Continuous Delivery (CI/CD) credentials used to push unauthorized changes (Ohm et al., 2020; OpenSSF, 2024). Security teams also deploy Integrated Development Environment (IDE) plugins and security context tools integrated into Artificial Intelligence (AI) coding assistants such as Claude Code or GitHub Copilot to catch problems before code reaches version control (GitHub, 2025a; Semgrep, 2024; Snyk, 2024).

In large enterprises, dozens of such tools produce findings through different data formats and integration patterns (Anderson, 2020; Stallings & Brown, 2024). As application security architect in a large organization with thousands of developers and tens of thousands of repositories, I have seen firsthand how hard this environment is to secure. Two classes of difficulty dominate.

The first is **detection diversity**. Tools in the same category scan differently and produce different results, so a single tool is rarely sufficient (Chess & West, 2007; McGraw, 2006). Running multiple tools in parallel improves coverage but creates the opposite problem: the same vulnerability is reported several times, and findings must be deduplicated before they can be triaged or counted (Gardner et al., 2023; Jacobs et al., 2020).

The second is **data consolidation**. Findings lack the business context needed for prioritization: the owning application, its criticality tier, and its regulatory scope live in a separate Configuration Management Database (CMDB), not in the security tool (AXELOS, 2019; Williams & Gonzalez, 2021). Correlation within the findings themselves is nontrivial: SAST results point to a repository, file, and line, while DAST results point to a Uniform Resource Locator (URL) or host, so cross tool correlation requires additional deployment and inventory metadata (Chess & West, 2007; Stuttard & Pinto, 2011). Integration patterns compound the problem. Push based models, where scanners upload reports into a vulnerability management interface, fragment data and do not scale across dozens of tools, and robust pull based Application Programming Interface (API) connectors, where they exist, are usually

gated behind commercial tiers (DefectDojo, 2024; Gardner et al., 2023).

Consolidating application security data is a data engineering problem. Industry addresses it with commercial Application Security Posture Management (ASPM) products offering dashboards and aggregated views (Gardner et al., 2023), but these are proprietary and inflexible. They create vendor lockin and prevent security teams from customizing pipelines, applying their own Machine Learning (ML) models, or integrating with internal systems.

The most notable open source alternative, DefectDojo, is a Django based vulnerability management web application (DefectDojo, 2024), not a data first platform. Its relational backend limits scale, and ingestion relies mostly on push based integrations or manual uploads. Pull based API connectors exist only in the commercial DefectDojo Pro tier.

Databricks Lakewatch, announced in March 2026, is a lakehouse native Security Information and Event Management (SIEM) (Databricks, 2026a). It runs on the same lakehouse platform this thesis adopts but addresses a different domain: runtime threat detection, alert triage, and log analytics, not application security posture. The distinction is discussed in Section 1.7.2.

What is missing is a vendor agnostic framework that approaches application security as a data engineering problem, providing a reusable architecture for ingestion, normalization, deduplication, and serving at enterprise scale. The pace of tool adoption outstrips the engineering effort available to write connectors for each one. A framework that encodes integration logic declaratively, as schemas, a connector contract, and a catalog of skills an AI coding agent can execute, **should reduce onboarding a new source to a four step procedure** (Section 3.1) rather than a pipeline redesign, with the cost per source bounded by the cost of invoking the skill catalog under supervision. Chapter 3 and Section 3.6.3 examine how far this reduction holds under the empirical methodology actually exercised.

This raises the central research question: **how can diverse enterprise application security findings be consolidated into a unified, vendor agnostic data platform that serves the analytical and operational needs of security teams?**

Objective

The main goal is to specify, design, and implement a data integration framework for enterprise application security that consolidates findings from tools into a unified platform, enabling consistent normalization, deduplication, triage, and correlation across business applications.

The subgoals are:

1. **Analysis** (Chapter 1): Survey the application security domain and its adjacent data sources. Characterize asset inventories, software development platforms, static and

dynamic security tooling, and the relevant data engineering patterns. Review related work, identify the research gap, and select source systems to carry forward. The API analysis per source grounding these findings is collected on the external documentation site.

2. **Framework** (Chapter 2): Formalize the requirements specification and propose a reusable blueprint covering solution architecture, a medallion based data model (bronze, silver, gold) (Strengholt, 2025), a connector framework, and an analytics and serving framework. The blueprint targets the Databricks lakehouse (Databricks, 2025d, 2025f; Lee et al., 2024) while separating general patterns from platform specific choices. The technology rationale is given in Section 2.1.1.
3. **Implementation** (Chapter 3): Produce a reference implementation on Databricks that instantiates the framework for the nine selected sources, all delivered as production ready connectors. The methodology drives generation through a chain of three skills (analyze-source, generate-connector, validate-implementation) executed against every source under reviewer subagent supervision. Section 3.6.3 grades the outcome. The deliverable is a minimum viable product that instantiates the framework end to end on a representative sample chosen to cover the ingestion and integration patterns the framework must support, rather than an exhaustive integration of every security tool an enterprise might deploy. Validation runs through automated tests with requirement traceability and through Lakeflow Declarative Pipelines (LDP) data quality expectations.

The scope is application security posture across three tiers of detection. **Static testing** covers preexecution analysis of source, dependencies, configuration, and build artifacts: SAST, SCA, secret scanning, container and Infrastructure as Code (IaC) scanning, and supply chain integrity checks. **Dynamic testing** covers active probing of running applications: DAST (OWASP ZAP (OWASP Foundation, 2024d)) and penetration testing. **Runtime security** covers observational and protective telemetry from production systems. One representative source is included, AWS WAF sampled requests (Amazon Web Services, 2026), to demonstrate that the correlation model links runtime findings to applications via deployment metadata. Broader runtime security operations (general threat detection, incident response, log analytics) remain out of scope. In all three tiers, findings are correlated to business applications, owners, and criticality.

Chapter 1 and Chapter 2 are framed against the full ASPM scope. Chapter 3 instantiates the framework against the nine sources selected in Section 1.8 as production ready connectors. ServiceNow and GitHub are reproduced in Section 3.4 as the two ends of the category range from source API to silver. Onboarding a tenth source repeats the procedure per source described in Section 3.1 and does not require changes to the framework.

The intended audience is application security architects and data platform engineers who need to consolidate security findings at scale without vendor lockin.

Methodology

This thesis follows the design science research methodology of Hevner et al. (2004) and Peffers et al. (2007) in three phases aligned with the three main chapters. The **analysis** phase combines a literature review with a systematic study of source systems, then identifies the research gap. The **framework** phase derives requirements and designs the architecture, data model, connector contract, and analytics and serving patterns. The **implementation** phase targets an AI instantiable reference implementation, generated end to end against all nine sources by a chain of three skills (analyze-source, generate-connector, validate-implementation) executed against each source. Each skill pass is then reviewed by a separate subagent (one for spec compliance, one for code quality), whose findings are folded back as follow up commits before the next source moves forward. Chapter 3 records what the chain produced per source and Section 3.6.3 grades the outcome. Validation runs through automated tests with requirement traceability and through LDP data quality expectations.

The thesis delivers three contributions: a **requirements specification**, a **reusable framework** covering architecture, data model, connector contract, and analytics and serving patterns, and an **AI instantiable reference implementation** on Databricks.

Structure

Chapter 1 surveys the problem domain and closes with related work, a gap analysis, and source system selection. Chapter 2 presents the framework: solution architecture, data model, connector framework, and analytics and serving framework. Chapter 3 reports on the MVP reference implementation, demonstrably generated end to end against all nine selected sources by a chain of three skills with reviewer subagent supervision. All nine land as production ready connectors. The Conclusion evaluates outcomes against the objectives, defends the AI instantiability claim in Section 3.6.3, discusses limitations, and suggests future work. Appendix A discloses the use of generative AI tooling. Figure 1 summarizes the contribution flow.

External Documentation Site

The full requirements specification is published as an external documentation site rather than inline in this thesis. It is structured into four sections (Requirements, Functional Specification, Design, Tests) generated with MkDocs Material from `mkdocs/docs/` inside the `appsec-mvp` repository and deployed via GitHub Pages. Normative language (SHALL, SHOULD) is preserved on the site.

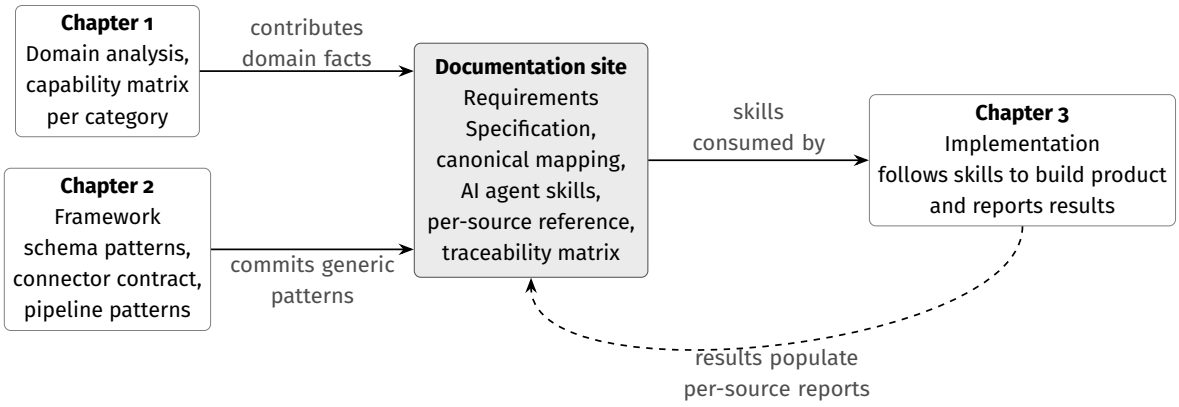

Figure 1

Thesis contribution flow. Chapter 1 (Analysis) and Chapter 2 (Framework) each contribute material to the Requirements Specification published at `https://vkraus.github.io/appsec-mvp/`. *Chapter 3 (MVP Implementation) consumes the specification. The skill chain ran end to end against the nine selected sources, with reviewer subagent cycles catching named defect classes evaluated in Section 3.6.3. Implementation results populate the Implementation reports per source on the specification (dashed return arrow).*

Documentation site: `https://vkraus.github.io/appsec-mvp/`
Source: `https://github.com/vkraus/appsec-mvp/tree/main/mkdocs`

Where the main chapters name a page on the documentation site, they link to it directly via an inline hyperlink. The top level sections of the site relevant to the thesis are the per category capability matrix, the documented mapping requirements, the connectors reference hub, and the requirement catalog and traceability matrix.

Offline copies of both the reference implementation and the documentation site are attached to this submission as `mvp.zip` (source code) and `docs.zip` (MkDocs sources and a prerendered static site under `docs/site/index.html`). The attachments are snapshots of the repository at the commit from which this Portable Document Format (PDF) was built. The live version remains at the URL above.

1. Analysis

This chapter analyzes the functional domains relevant to building an application security data platform. Each section combines theoretical foundations with practical tool and API analysis, focusing on source system interfaces, data models, and integration patterns. Technology choices are deferred to Chapter 2 and Chapter 3.

1.1 Asset Discovery and Inventory

Asset discovery and inventory is the foundational data source for any application security data platform. Organizations maintain application inventories in a CMDB or custom catalog, while cloud providers and container orchestrators expose asset inventories through APIs. Without this context, a vulnerability cannot be attributed to an owner, scored for business impact, or prioritized.

1.1.1 Application Portfolio Management

Enterprise organizations maintain catalogs of their business applications with metadata such as application name, description, ownership information, lifecycle status, and relationships to other assets (AXELOS, 2019).

These registries also capture security relevant attributes. Information security is commonly assessed along the Confidentiality, Integrity, and Availability (CIA) triad: confidentiality (preventing unauthorized disclosure), integrity (preventing unauthorized modification), and availability (ensuring authorized access) (Stallings & Brown, 2024). Business impact assessments evaluate all three CIA dimensions to produce criticality ratings, commonly structured as tiers: Tier 1 for mission critical applications, Tier 2 for important supporting systems, and Tier 3 for noncritical internal tools (National Institute of Standards and Technology, 2004). Regulatory scope flags identify which compliance frameworks apply to a given application.

For data integration, the most important attribute is the mapping between business applications and their technical assets. A single business application may span multiple repositories, microservices, infrastructure components, and deployment environments. When the framework ingests a vulnerability from a SAST scanner linked to a specific repository, this mapping determines which business application is affected, who owns it, and how critical it is, so that prioritization reflects risk.

1.1.2 Configuration Management Databases

The most common CMDB platform in enterprise environments is ServiceNow, holding over 44% of the global IT Service Management (ITSM) market share (Gartner, 2024). It serves as the reference platform for this thesis. A CMDB organizes all managed assets as Configuration Items (CIs), each with attributes describing its type, status, and ownership (AXELOS, 2019).

ServiceNow arranges CIs in a class hierarchy rooted at the `cmdb_ci` base table. Each subclass inherits base attributes (name, `sys_id`, operational status, environment) and adds domain specific fields. For application inventory purposes, the most relevant class is `cmdb_ci_business_app` (Business Application), which extends the base CI with attributes such as version, business unit, used for classification, and references to the owning support group (ServiceNow, 2024a). The Minimum Viable Product (MVP) pipeline ingests this class together with `cmdb_rel_ci`.

Beyond the application record itself, the CMDB captures relationships between CIs through a dedicated relationship table (`cmdb_rel_ci`). These model dependencies such as "runs on" (application to server), "depends on" (application to database), and "hosted on" (application to cloud service). The most valuable relationships for the framework are those linking business applications to technical assets corresponding to entities in the security data: repository names, deployment targets, and infrastructure components. ServiceNow allows custom attributes and relationship types, so available fields vary between deployments.

1.1.3 Integration Options

Integrating CMDB data requires choosing between two strategies: pulling data from ServiceNow via its APIs, or pushing data from a custom ServiceNow application to an external target.

1.1.3.1 ServiceNow APIs

ServiceNow exposes two complementary Representational State Transfer (REST) APIs for CMDB data extraction. The Table API provides generic CRUD operations against any ServiceNow table through the endpoint `/api/now/table/{tableName}` (ServiceNow, 2024d). Responses are JavaScript Object Notation (JSON) formatted, with support for server side filtering (`sysparm_query`), field selection (`sysparm_fields`), and pagination via `sysparm_limit` and `sysparm_offset` (default maximum of 10 000 records per request). The `sysparm_display_value` parameter resolves reference fields to human readable names instead of internal `sys_id` values.

The CMDB Instance API (`/now/cmdb/instance/{className}`) provides a CMDB aware alternative that understands the CI class hierarchy (ServiceNow, 2024a). It retrieves a CI with its

relationships in a single call, reducing API calls needed to reconstruct the mapping from application to asset, but is more constrained in query capabilities than the Table API.

Both APIs require authentication, typically Basic Authentication with a service account granted the `rest_service` or `cmdb_read` role, or OAuth 2.0 for more security conscious deployments (ServiceNow, 2024b). Rate limits are governed by platform transaction quotas rather than per endpoint limits. For incremental extraction, the `sys_updated_on` timestamp present on all records serves as a reliable high water mark (ServiceNow, 2024d).

Any pull based integration must handle authentication, pagination, rate limiting, and incremental state management. Tooling choices specific to each source are documented on the ServiceNow source reference page.

1.1.3.2 ServiceNow Application

An alternative is to push data from the source. ServiceNow supports scoped applications running on the platform. A custom application can use Outbound REST Messages to send CMDB data to an external target (a cloud storage bucket or a REST endpoint) on a schedule defined through Scheduled Jobs or Flow Designer. It can also expose a Scripted REST API (ServiceNow, 2024c): a custom endpoint that formats data exactly as the consumer needs it, prejoining application records with relationships and custom attributes in a single response.

The advantage is full access to ServiceNow's scripting, relationship traversal, and business logic without external API call overhead. The disadvantage is that development and maintenance happen inside ServiceNow, requiring platform expertise and a separate deployment lifecycle, with schema or logic changes coordinated across two platforms. This approach suits organizations with mature ServiceNow development teams that prefer to own export logic on the source side.

1.1.3.3 Integration Challenges

The application inventory is the least standardized integration among all data sources. Security testing tools converge around common API patterns and formats, but each organization structures its application inventory differently: hierarchy, granularity of asset mapping, and captured attributes vary substantially (AXELOS, 2019). This makes the inventory connector hardest to generalize and most likely to require customization.

Data quality in CMDBs is a persistent concern (Williams & Gonzalez, 2021). Records are often incomplete or stale. A common case is a decommissioned application still listed as active with no current owner. Integrations must account for these issues through validation rules and defaults that flag problems without blocking the pipeline.

Despite these difficulties, the application inventory is indispensable: without its business context, findings lack organizational meaning for prioritization and governance.

Alternative platforms include BMC Helix, Device42, and the open source Ralph. Cloud native services such as AWS Config and Azure Resource Graph expose equivalent inventories for cloud resources, consumed through provider native APIs with IAM scoped credentials. The bronze ingestion pattern (Chapter 2) applies to any source with an inventory API.

1.2 Software Development

Modern software is built collaboratively from source code, organized in repositories, and deployed through automated pipelines (Pressman & Maxim, 2019; Sommerville, 2015). Repositories, build pipelines, and issue trackers form the second major data source category for the framework. These platforms share a common integration pattern: REST over JSON, authentication via OAuth tokens, Personal Access Tokens (PATs), or service accounts, offset or cursor pagination, and rate limits that require backoff. This consistency enables shared connector logic.

1.2.1 Source Code Management

Chacon and Straub (2014) provide a thorough treatment of Git, covering branching, merging, and distributed workflows. Skoulikari (2024) offers a more recent visual introduction. Winters et al. (2020) extend this to organizational scale, describing how large teams manage repositories with thousands of contributors. For this thesis, the repository is the primary entity around which findings are grouped, as detailed in Chapter 2.

Development teams host Git repositories on cloud based Source Code Management (SCM) platforms. GitHub is dominant, with over 150 million developers and adoption by 92% of Fortune 100 companies (GitHub, 2025b). The 2025 Stack Overflow Developer Survey reports GitHub at 81% adoption for code collaboration, followed by GitLab at 36% (Stack Overflow, 2025). Bitbucket and Azure DevOps serve smaller but significant enterprise segments. Most large organizations run multiple platforms through acquisitions, team preferences, or regulatory splits. Multi platform ingestion is therefore a practical requirement.

Beyond source code, SCM platforms expose security relevant metadata: primary language, creation and last activity dates, visibility, and archive status. Branch protection rules indicate development process maturity. Team and contributor assignments establish ownership feeding into the mapping from application to team from Section 1.1.

GitHub exposes two APIs (GitHub, 2024a): REST (v3) with resource oriented endpoints and page based pagination, and GraphQL (v4) with selective field retrieval and cursor pagination. Both share authentication (OAuth tokens or PATs) and a rate limit of 5 000 requests per

hour (GitHub, 2024b). All use token based authentication and JSON responses, enabling shared connector logic.

Other enterprise SCM platforms expose comparable REST (and sometimes GraphQL) APIs: GitLab, Bitbucket, and Azure DevOps Repos.

1.2.2 Continuous Integration and Delivery

Software Development Lifecycle (SDLC) practices have shifted from sequential models to iterative methodologies and DevOps. Forsgren et al. (2018) present empirical evidence that high performing teams deploy more frequently with shorter lead times through CI/CD automation. Kim et al. (2021) provide practical guidance for implementing DevOps pipelines at scale. The CI/CD pipeline is relevant because each stage can integrate security tools.

GitHub Actions is the most adopted CI/CD platform, used by 62% of developers for personal projects and 41% in organizational settings (JetBrains, 2025). Jenkins remains prevalent in enterprises at 28% organizational adoption, followed by GitLab CI at 19%. 32% of organizations use two or more CI/CD tools and 9% use at least three, reinforcing the multiplatform integration challenge seen with SCM platforms.

CI/CD platforms provide contextual data about security scans. Pipeline definitions reveal which tools run. Build results show whether gates passed. Execution metadata records commit, timing, and duration. This is operational context rather than a primary finding source: a repository whose last scan ran three months ago, or whose scans consistently fail, signals coverage gaps and tool health.

Integration patterns vary more across CI/CD platforms than across SCM tools. GitHub Actions and Azure Pipelines expose REST APIs with JSON responses, Jenkins uses an Extensible Markup Language (XML) based API with a different authentication model, and GitLab CI piggybacks on the GitLab API. Despite these differences, retrieved data is structurally similar: pipeline identifiers, run statuses, timestamps, and commit references. None of these CI/CD platforms is built as a connector in the reference implementation.

1.2.3 Issue Tracking

Issue trackers record remediation status. When a finding is assigned for remediation, an issue is created in the tracker for the team manually or through automation. The framework reads issue status to determine whether a finding is under active remediation, who is responsible, and whether it has been resolved within the Service Level Agreement (SLA) window.

Jira is the dominant enterprise issue tracker. The 2025 Stack Overflow Developer Survey reports Jira at 46% adoption, behind only GitHub (81%) and ahead of GitLab (36%) (Stack

Overflow, 2025). Azure DevOps Work Items serves the Microsoft ecosystem. GitHub Issues and GitLab Issues provide lightweight built in tracking tied to their SCM platforms. Many organizations use multiple trackers across teams.

These platforms expose REST APIs with JSON responses and paginated results, and most support webhooks for real time status change notifications. None is built as a connector in the reference implementation.

Integration is bidirectional: the framework reads remediation status and may create new issues when findings require attention. This raises idempotency concerns (avoiding duplicate issue creation) and state synchronization (keeping the view of the framework consistent with the tracker).

1.3 Static Application Security

Application security encompasses practices and tools for identifying vulnerabilities throughout the software lifecycle (Anderson, 2020; Stallings & Brown, 2024). McGraw (2006) advocated for security touchpoints spanning development. The DevSecOps paradigm (Kim et al., 2021; Mack, 2023) realizes this by embedding security testing into CI/CD pipelines. SAST examines source code, SCA checks third party dependencies, secret scanners flag leaked credentials, and DAST probes running applications. These tools produce large volumes of data in different APIs, formats, and severity models. The Open Worldwide Application Security Project (OWASP) Top 10 classifies common web application risks (OWASP Foundation, 2021) but does not define a data interchange standard.

Several cross cutting standards bring partial consistency. The Common Vulnerabilities and Exposures (CVE) system assigns unique identifiers to known vulnerabilities (MITRE Corporation, 2024a). Common Weakness Enumeration (CWE) classifies weakness types (MITRE Corporation, 2024b). Common Vulnerability Scoring System (CVSS) scores attack vector, complexity, and impact on a 0–10 scale (Forum of Incident Response and Security Teams, 2019). Exploit Prediction Scoring System (EPSS) adds a predictive signal (Jacobs et al., 2020): the ML computed probability that a vulnerability will be exploited within 30 days. Cybersecurity and Infrastructure Security Agency (CISA) maintains the Known Exploited Vulnerabilities (KEV) catalog (Cybersecurity and Infrastructure Security Agency, 2024), which adds a third signal: confirmed active exploitation from real world incidents.

For data interchange, Static Analysis Results Interchange Format (SARIF) defines a JSON based format for static analysis results (OASIS Open, 2024). CycloneDX and Software Package Data Exchange (SPDX) provide Software Bill of Materials (SBOM) schemas (Linux Foundation, 2024; OWASP Foundation, 2024a). Open Cybersecurity Schema Framework (OCSF) standardizes security event exchange for SIEM and Security Orchestration, Automation, and Response (SOAR) use cases (OCSF Project, 2024). No single format covers all tool categories:

- SARIF addresses static analysis but not SCA or runtime findings.
- CycloneDX and SPDX cover software composition, not code level vulnerabilities.
- OCSF serves security operations, not application security posture.

This gap motivates the normalization model in Chapter 2.

Static analysis tools examine source code and build artifacts without executing the application. Their findings point to repositories, files, and lines, so developers own them directly. Mobile application security testing is excluded, as the framework targets web applications and backend services.

1.3.1 Static Application Security Testing

SAST examines source code, bytecode, or binary code without executing the program. Chess and West (2007) cover taint analysis, control flow analysis, and pattern matching for detecting injection flaws, buffer overflows, and insecure data handling. Broader rule sets catch more issues and generate more noise (Chess & West, 2007). That is the central SAST tradeoff.

SonarQube exposes a REST Web API (source reference), and Semgrep runs as a Command Line Interface (CLI) tool producing JSON or SARIF output with an optional Cloud Platform API (source reference). SCM integrated scanners such as GitHub Code Scanning (CodeQL) and GitLab SAST report through the host SCM API. Commercial alternatives include Checkmarx and Fortify, both exposing REST APIs. When platform native and standalone tools scan the same repository, the overlap requires deduplication in the normalization layer.

Findings typically include a rule identifier, affected file path and line number, severity, code snippet, and remediation guidance. Severity models differ. SonarQube uses a five level scale (blocker, critical, major, minor, info). Semgrep and Checkmarx use high, medium, and low, sometimes with a numerical score. Severity normalization is therefore a core transformation. Chapter 3 resolves SonarQube and Semgrep to ingestion categories and publishes their reference pages per source. Only the ServiceNow and GitHub connectors are built as working code at MVP time.

1.3.2 Software Composition Analysis

SCA identifies known vulnerabilities in third party dependencies. Modern applications pull in hundreds of transitive dependencies (Anderson, 2020), any of which can introduce a vulnerability. Ohm et al. (2020) provide a systematic review of supply chain attacks including typosquatting, dependency confusion, and malicious package injection. SCA tools analyze dependency manifests, resolve the full tree, and cross reference versions against vulnerability databases such as the National Vulnerability Database (NVD).

OWASP Dependency-Track is an open source platform that ingests SBOMs in CycloneDX or SPDX format and exposes a REST API (source reference). Snyk exposes a REST API, and Dependabot exposes alerts through the GitHub GraphQL API. Many SCA tools also generate SBOMs (Linux Foundation, 2024; OWASP Foundation, 2024a), increasingly required by regulators (National Telecommunications and Information Administration, 2021). Chapter 3 resolves Dependency-Track to data load tool (dlt), and GitHub Dependabot and GitLab dependency scanning are reached through their respective platform APIs. Only the ServiceNow and GitHub connectors are built as working code at MVP time.

SCA output is more standardized than other categories. The data comes from shared public databases (CVE, NVD). Findings include the vulnerable dependency name and version, a CVE identifier, a CVSS score, the fixed version when available, and exploitability metadata. Tools may resolve the same dependency tree differently, so two scanners flag inconsistent CVEs. The framework handles this during deduplication, recognizing that the same CVE reported by multiple tools against the same repository likely represents a single issue.

SCA tools also detect license violations (e.g., GPL licensed code in proprietary applications). The Supply-chain Levels for Software Artifacts (SLSA) framework (OpenSSF, 2024) complements SCA. SCA checks whether dependencies contain known vulnerabilities. SLSA verifies the integrity and provenance of build artifacts.

OWASP Dependency-Check, a CLI oriented alternative that writes JSON or XML reports from CI/CD runs, fits the same CLI artifact ingestion pattern as Semgrep's CLI mode.

1.3.3 Secret Detection

Secret detection tools scan version control history for credentials, API keys, tokens, and other sensitive material. Once committed, a secret persists in Git history even after deletion from the working tree. Two techniques apply: regex patterns against known credential formats, and entropy scoring against unusually random strings. The entropy approach catches novel formats at higher false positive rates.

TruffleHog is a CLI tool combining pattern and entropy analysis with live credential verification, producing JSON output parsed as an artifact (source reference). GitLeaks and detect-secrets are similar CLI oriented alternatives, and commercial offerings such as GitGuardian expose REST APIs for centralized management.

Scanners integrated into the platform differ. GitHub Secret Scanning exposes alerts through the GitHub REST API with webhook notifications (GitHub, 2024a), and GitLab Secret Detection reports through GitLab's security dashboard and API. These are simpler to integrate because they reuse platform APIs but cover only repositories on their own platforms.

There is no standard output format. Each tool defines its own JSON schema, and none supports SARIF. Findings include secret type, file path, commit reference, and sometimes

validity status, but field names and structures vary.

1.4 Dynamic Application Security Testing

Dynamic application security testing operates on running applications by actively issuing requests and evaluating responses. Unlike static analysis, it requires a deployed instance and detects flaws that manifest only at execution time: authentication bypass, server misconfiguration, and certain injection classes. This section covers active scanning. Section 1.5 covers passive telemetry. The cross cutting standards from Section 1.3 apply equally to dynamic findings.

1.4.1 Dynamic Application Security Testing

DAST tests running applications by sending crafted Hypertext Transfer Protocol (HTTP) requests and analyzing responses for vulnerability indicators (Stuttard & Pinto, 2011). Unlike SAST, DAST requires a deployed application and detects vulnerabilities that manifest only at runtime, such as authentication bypass, server misconfiguration, and certain injection flaws.

OWASP Zed Attack Proxy (ZAP) is the most widely used open source DAST tool, exposing a REST API for scan management and alert retrieval (source reference). Burp Suite Enterprise exposes a REST API, while the Professional edition is a desktop tool without programmatic access. Findings include the target URL, affected HTTP parameter, vulnerability type mapped to CWE (MITRE Corporation, 2024b), exploitation evidence, and confidence rating.

The core integration challenge is that DAST findings are URL based, not code based. Mapping URL based findings to source repositories requires deployment metadata or inventory data from Section 1.1. Without this, DAST findings can be attributed to applications but not to development teams or code locations.

1.5 Runtime Security

Runtime security covers passive telemetry from production workloads, sitting alongside the active scanning in Section 1.4 and completing the three detection tiers the framework uses: static, dynamic, and runtime. The representative tool in this tier is the Web Application Firewall (WAF). It does not execute scans. It inspects live HTTP traffic and emits events when rules fire, marking exploitation attempts against a workload or endpoint and linking to applications through deployment metadata rather than source code coordinates. Broader runtime operations such as SIEM driven incident response and general log analytics are

adjacent but out of scope. The framework treats runtime security as one input tier among three, correlated with static and dynamic findings through the shared application inventory.

1.5.1 Web Application Firewalls

WAFs operate at the network perimeter, inspecting inbound HTTP traffic against rule sets such as the OWASP Core Rule Set (OWASP Foundation, 2024c). They log blocked and flagged requests with matched rule, request details, and action taken. Amazon Web Services (AWS) WAF exposes the `GetSampledRequests` API as the primary interface, with CloudWatch Logs as an opt in extension for deployments requiring full fidelity logging (source reference). Cloudflare and Akamai expose comparable REST APIs for log retrieval and configuration.

These platforms bundle Distributed Denial of Service (DDoS) protection alongside WAF capabilities. DDoS mitigation logs traffic patterns, attack signatures, and mitigation actions, providing availability impact data that complements vulnerability findings.

WAFs emit event streams, not discrete findings. The framework consumes aggregated patterns, not raw logs.

1.5.2 Source Integration Summary

The source characteristics reference page tabulates integration characteristics across the AppSec source landscape surveyed for the thesis.

Several patterns emerge. REST APIs with JSON responses are dominant, but XML, GraphQL, CLI output parsing, and manual uploads are all necessary. Standardization varies: SCA benefits from the CVE/NVD ecosystem, SAST is partially standardized through SARIF, and secret scanning has no standard format. Update frequencies range from continuous dependency monitoring to periodic penetration tests. These characteristics define what the ingestion layer in Chapter 2 must accommodate.

1.6 Data Engineering

Consolidating the diverse sources analyzed above into a unified analytical platform is a data integration problem. This section surveys the architectural paradigms and engineering patterns relevant to solving it.

1.6.1 Data Platform Architecture

The literature documents a progression from data warehouses through data lakes to the lakehouse. DAMA International (2017) provide the data management body of knowledge. The data warehouse, formalized by Inmon (2005), stores subject oriented, integrated data using a normalized approach. Kimball and Ross (2013) take a bottom up path with dimensional modeling: star schemas of fact and dimension tables. Both approaches assume structured, stable sources. Security tool output is neither.

The data lake (Miloslavskaya & Tolstoy, 2016) stores raw data in its native format, deferring schema to consumption time (schema on read). This accommodates format diversity but risks "data swamps" without governance (DAMA International, 2017).

The lakehouse (Armbrust et al., 2021) combines data lake flexibility with warehouse governance through open table formats adding Atomicity, Consistency, Isolation, Durability (ACID) transactions, schema enforcement, and time travel on top of lake storage (Lee et al., 2024; Thalpati, 2024). It accepts semi structured tool output and gives the governance and fast queries needed for both dashboards and operational lookups.

The medallion architecture (Strengholt, 2025) organizes lakehouse data into three layers:

- **Bronze**: Raw data with minimal transformation, preserving original source schemas.
- **Silver**: Cleaned, validated, and normalized data conforming to a unified schema.
- **Gold**: aggregated metrics and enriched datasets ready for queries.

This maps naturally to security data integration. Bronze captures raw tool output in source native schemas, silver normalizes findings into a vendor agnostic model, and gold computes aggregated risk metrics and enriched datasets stakeholders consume.

A security data platform must serve two access patterns. Online Analytical Processing (OLAP) queries power dashboards and trend analysis over large aggregated datasets. Online Transaction Processing (OLTP) access supports operational workflows such as issue tracker integration and real time risk lookups, where low latency reads and writes to individual records are essential. Both inform the serving architecture in Chapter 2.

1.6.2 Data Integration Patterns

Reis and Housley (2022) distinguish Extract, Transform, Load (ETL) from Extract, Load, Transform (ELT). ETL transforms data before loading, requiring upfront schema knowledge. ELT loads raw data first and transforms in place, aligning with lakehouse architectures where distributed compute engines perform transformations at scale (Thalpati, 2024). Security tool output varies and evolves, so upfront transformation breaks. ELT fits.

Full reingestion does not scale across thousands of repositories and dozens of tools.

Incremental ingestion pulls only new or changed records. The high water mark pattern tracks a timestamp or cursor per source. Change Data Capture (CDC) captures changes at the source level, propagating inserts, updates, and deletes as events (Reis & Housley, 2022).

Source APIs and formats change regularly, so rigid schemas break pipelines on every upstream change. Schema on read stores raw data in its original structure and applies schema at query time. Additive evolution accepts new fields (Lee et al., 2024), and existing queries keep working.

Reruns are idempotent so retries and replays do not duplicate. Each layer validates: schema at ingest, value ranges at normalization, referential integrity at aggregation. Records that fail validation go to quarantine tables (DAMA International, 2017).

1.6.3 Domain Data Model

The security domains analyzed above produce data about a common set of entities and relationships. A vendor agnostic conceptual model must precede physical schemas. The domain model has five primary entities:

- **Applications** carry business context from asset inventories (Section 1.1): name, owning team, criticality tier, lifecycle status, and compliance scope. An application represents a business level unit that may span multiple repositories and deployment environments.
- **Repositories** represent technical assets from SCM platforms (Section 1.2): code location, primary language, visibility, activity indicators, and archive status. The repository is the primary unit around which security findings are grouped.
- **Findings** capture security issues from any tool category analyzed in Section 1.3 and Section 1.4: severity, status, source tool, detection timestamp, affected code location, and rule or check identifier. Each finding traces to a specific tool scan.
- **Vulnerabilities** are CVE identified issues enriched with three signals: CVSS severity from the NVD, exploitation probability from EPSS, and confirmed exploitation status from CISA KEV. Multiple findings from different tools may reference the same vulnerability.
- **Teams** provide organizational ownership, linking personnel to applications and remediation responsibilities. Team structure enables filtering and aggregation by organizational unit.

More entities add development and supply chain context:

- **Commits** record individual code changes with author, timestamp, and affected files. They link findings to specific code versions and establish authorship for attribution.
- **Pull requests** represent code change proposals where security scans typically execute. Scan results are often reported per pull request, making it a natural grouping unit for new findings.
- **Pipeline runs** capture CI/CD execution records: which security tools ran, when, against

which commit, and whether security gates passed. They indicate scan coverage and tool health across repositories.

- **Issues** track remediation work items in systems such as Jira. Linking findings to issues enables Mean Time to Remediate (MTTR) calculation and SLA compliance monitoring.
- **Dependencies** represent third party libraries identified through SCA: package name, version, license, and associated CVE identifiers. They connect SCA findings to the specific library version that introduced the vulnerability.
- **Branch protection policies** capture governance configurations from SCM platforms: required reviewers, mandatory status checks, and merge restrictions. They serve as indicators of development process maturity and security hygiene at the repository level.

Figure 1.1 illustrates the entity relationships. Primary entities (bold borders) form the core analytical model. Supporting entities provide development process and supply chain context.

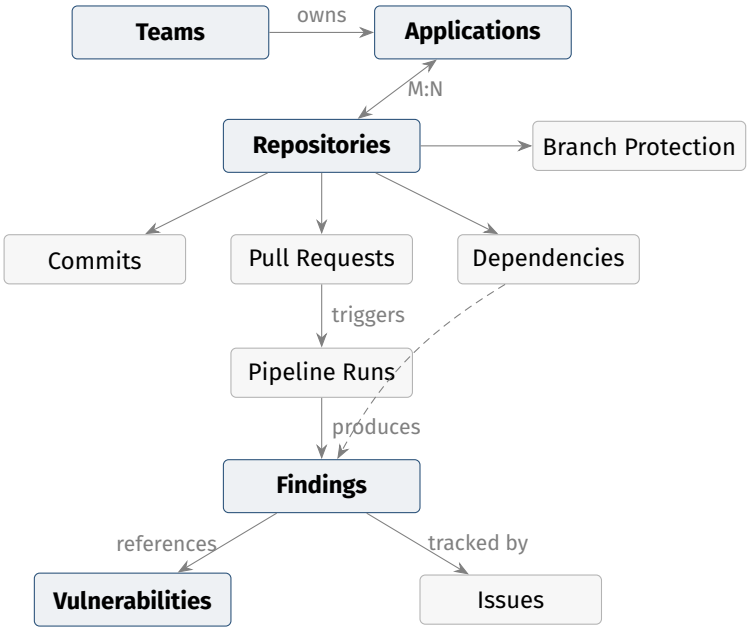

Figure 1.1
Conceptual Domain Model

The mapping from app to repo is the key relationship. It links findings to business context and is many to many: shared libraries serve multiple applications, and microservice applications span multiple repositories. Teams own applications, establishing the chain from organizational accountability through business applications to technical assets and findings. Dependencies bridge repositories and vulnerabilities: an SCA finding links a specific library version to a known CVE. Issues track remediation.

The structure maps to dimensional modeling (Kimball & Ross, 2013). Findings are facts. Applications, repositories, and teams are dimensions. Commits, pull requests, and pipeline runs add temporal and process dimensions. Cross tool deduplication collapses related

findings while preserving traceability to each source.

Stakeholders consume this data through different patterns:

- Application owners need aggregated risk dashboards showing finding counts by severity, remediation progress, and compliance status.
- Developers need actionable findings with code level context, delivered through issue trackers.
- Security experts need drill down access to raw findings, audit trails, and cross tool correlation for triage.
- Leadership needs executive metrics: MTTR, SLA compliance, vulnerability density trends, and portfolio level comparisons.

These span the OLAP and OLTP requirements from Section 1.6.1 and drive the serving architecture in Chapter 2.

1.7 Related Work and Gap Analysis

No standard approach exists for consolidating security tool output into a unified data platform. Organizations face a fragmented landscape of partial solutions.

1.7.1 Existing Approaches

Gardner et al. (2023) define ASPM as tools that continuously manage application risk through collection, analysis, and prioritization of security issues across the software lifecycle. Commercial platforms such as Apiiro, Cycode, ArmorCode, and Snyk AppRisk aggregate findings into unified dashboards with correlation and risk scoring, but they are proprietary. Pipelines cannot be customized, custom ML cannot be added, and integrations are limited to what the vendor supports.

Platform native features cover only their own ecosystem. GitHub Advanced Security provides code scanning via CodeQL, secret scanning, and dependency review (GitHub, 2025a). GitLab bundles SAST, DAST, dependency scanning, container scanning, and a security dashboard (GitLab, 2025). Organizations using multiple SCM platforms or third party tools cannot consolidate through them.

DefectDojo (DefectDojo, 2024) is the most prominent open source alternative, supporting imports from over 150 tools through format specific parsers. It was designed as a vulnerability management interface rather than a data platform. Its PostgreSQL backend limits enterprise scale analytics, and advanced API connectors are commercial only. OCSF (OCSF Project, 2024) standardizes security event schemas for SIEM and SOAR but does not model application level entities such as repositories, applications, or teams.

Many enterprises build ad hoc integrations: custom scripts, ETL pipelines, and dashboards from manual exports. These lack schema governance, break on API changes, and persist as institutional knowledge in unmaintained scripts.

1.7.2 Databricks Lakewatch

Databricks announced Lakewatch in March 2026 as an open, agentic SIEM built on the Databricks lakehouse (Databricks, 2026a). It unifies security, Information Technology (IT), and business data in a single governed environment powered by Delta Lake and Unity Catalog, with agentic triage, Detection as Code, and integrations from Wiz, Palo Alto Networks, Okta, and Zscaler.

Lakewatch focuses on runtime security operations: threat detection, incident response, alert triage, and log analytics. This framework focuses on application security posture: findings from SAST, SCA, DAST, secret scanning, container scanning, and IaC tools, combined with remediation tracking and risk aggregation. The two are complementary. Shared lakehouse infrastructure enables data exchange: Lakewatch could consume application risk scores for incident triage, and the framework could ingest Lakewatch's runtime detections as a finding source.

1.7.3 Comparative Analysis and Research Gap

Table 1.1 compares the surveyed approaches against criteria from the preceding analysis.

No existing approach satisfies all criteria. Commercial ASPM is proprietary. Platform native aggregations are ecosystem locked. DefectDojo lacks the data architecture for enterprise scale analytics. Lakewatch addresses runtime operations rather than application security posture. Academic work on application security consolidation as a data engineering problem is limited, focusing on detection techniques rather than cross tool integration at scale.

1.8 Selected Sources

The reference implementation in Chapter 3 instantiates the framework against nine source systems chosen to cover the ingestion and integration patterns that the framework must support, while keeping the sample tractable. The selection spans all three detection tiers: static testing, dynamic testing, and runtime security.

Table 1.1
Comparison of Existing Approaches

Criterion	ASPM	Platform native	DefectDojo	Lakewatch	This thesis
Open source	–	–	Yes	–	Yes
Vendor agnostic model	–	–	Partial	–	Yes
Extensible connectors	–	–	Yes	Partial	Yes
Lakehouse architecture	–	–	–	Yes	Yes
OLAP + OLTP serving	–	–	–	–	Yes
Data quality enforcement	–	–	–	–	Yes
AppSec specific model	Yes	Partial	Yes	–	Yes
Enterprise scalability	Yes	Partial	–	Yes	Yes

Note. Yes indicates full support, Partial indicates acknowledged but not demonstrated, – indicates out of scope or not addressed. Source: author synthesis of cited products' public documentation, accessed 2026-04.

1.8.1 Selection Criteria

The nine sources are selected based on five criteria:

1. **Open source or free tier available.** The reference implementation must be reproducible without commercial licenses.
2. **Full domain coverage across three detection tiers.** The nine together must cover every domain in the framework: CMDB, SCM, and the three detection tiers, namely static testing (SAST, SCA, secrets), dynamic testing (DAST), and runtime security (WAF).
3. **API heterogeneity.** The selection deliberately includes REST (ServiceNow, SonarQube, Dependency-Track, GitHub, GitLab, OWASP ZAP, AWS WAF), GraphQL (GitHub, GitLab), and CLI wrapped (Semgrep, TruffleHog) sources so the connector contract is exercised across integration styles.
4. **Limit heterogeneity to nine to maintain scope control.** The nine sources cover the ingestion and integration patterns that the framework must support. A wider sample would dilute pattern coverage without strengthening the framework claim.
5. **Industry adoption.** Each tool is in widespread enterprise use.

A summary of the variation in incremental strategies among the selected sources is presented in Table 1.2.

Table 1.2
Pairings of source and incremental strategy

Source	Incremental strategy
ServiceNow	Native `sys_updated_on` high water mark column
SonarQube, Semgrep Cloud, Dependency-Track	Native high water mark column (SonarQube `updateDate`, Semgrep Cloud `since_date`/`updated_at`, Dependency-Track `lastOccurrence`)
GitHub, GitLab	Webhook
Semgrep Docker, TruffleHog	Commit SHA
OWASP ZAP	Scan lifecycle retrieval
AWS WAF	Event time windowed sampled retrieval

Note. Classification synthesized from the medallion pattern treatment in Strengholt (2025) and the Lakeflow Declarative Pipelines operations guidance in (Databricks, 2025d). Pairings of source and strategy are author attributions based on the API capability matrix per source.

1.8.2 Selected Sources

- **ServiceNow**: CMDB. Dominant enterprise CMDB with a free developer instance, REST Table and CMDB Instance APIs, and native `sys_updated_on` high water mark.
- **GitHub**: SCM plus SAST, SCA, and secrets scanning integrated into the platform. REST and GraphQL APIs, rich webhook coverage, cursor pagination.
- **GitLab**: SCM plus security integrated into the platform. REST and GraphQL APIs, keyset and offset pagination, webhook coverage. Security findings require Ultimate tier or pipeline artifacts.
- **SonarQube**: SAST (static testing). Mature server product with a REST Web API, well documented severity scale, `updateDate` high water mark, and a scan completion webhook that complements scheduled polling.
- **Semgrep**: SAST (static testing). Modern engine with both a CLI deployment (CI/CD embedded) and a hosted Cloud Platform API. Exercises the dual CLI artifact and server API ingestion pattern.
- **Dependency-Track**: SCA (static testing). OWASP project with a REST API, SBOM centric (CycloneDX, SPDX) vulnerability correlation across multiple advisory sources.
- **TruffleHog**: secrets (static testing). CLI tool with no server API, distinguished by live credential verification that populates the canonical `validity_status` field.
- **OWASP ZAP**: DAST (dynamic testing). Open source dynamic scanner operated as a long running daemon with a REST API (OWASP Foundation, 2024d) for scan orchestration and alert retrieval. URL based findings represent the dynamic testing tier and exercise the on demand scan lifecycle integration pattern.
- **AWS WAF**: WAF (runtime security). Managed web application firewall exposing matched

request samples through the `GetSampledRequests` API (Amazon Web Services, 2026), with full event logs available via CloudWatch. Event based findings linked to workloads by Amazon Resource Name (ARN) and tag represent the runtime security tier and exercise the time window sampled retrieval pattern.

Details for each of the selected sources are on the connectors reference hub. Other tools named in this survey do not have dedicated reference pages.

1.8.3 Considered and Excluded

Table 1.3 enumerates the key tools that were considered and the criterion that excluded each. Tools marked **within scope, deferred** satisfy all criteria but are excluded based on the cap in Criterion 4. Onboarding them would simply be a repeat of the procedure per source in Section 3.1.

Table 1.3
Considered and excluded sources. Each row names the tool, the category it would have populated, and the criterion that excluded it (C1 open source or free tier, C2 full domain coverage, C3 API heterogeneity, C4 heterogeneity cap, C5 industry adoption).

Tool	Category	Excl. by	Note
Checkmarx	SAST	C1	commercial only, no permissive tier
Fortify	SAST	C1	commercial, OpenText
Burp Enterprise	DAST	C1	commercial. Community edition is manual only
Snyk	SCA / SAST	C4	**within scope, deferred** (overlap with Dependency-Track and Semgrep)
GitGuardian	secrets	C4	**within scope, deferred** (overlap with TruffleHog)
GitLeaks	secrets	C4	**within scope, deferred** (overlap with TruffleHog)
Dependabot	SCA	C4	**within scope, deferred**. GitHub native, overlap with Dependency-Track
Jira	issue tracker	C2	issue tracking is out of the ASPM data scope
Azure DevOps	SCM / CI/CD	C4	**within scope, deferred** (overlap with GitHub and GitLab)
Bitbucket Cloud	SCM	C4	**within scope, deferred**
Jenkins	CI/CD	C2	CI orchestrator, does not produce ASPM findings on its own
GitHub Actions	CI/CD	C4	**within scope, deferred**
Cloudflare WAF	WAF	C4	**within scope, deferred** (overlap with AWS WAF)
Akamai WAF	WAF	C1	commercial, no free tier

1.8.4 Cross Source Synthesis

Across the nine sources, four patterns recur: ticket like, resource like, finding like, and event like. Severity scales span three to six levels with overlapping but not identical vocabularies. Status vocabularies diverge more sharply, with scanner specific states that rarely map one to one with a common lifecycle. Five pagination strategies appear: offset (ServiceNow, SonarQube, Dependency-Track), cursor (GitHub REST, Semgrep Cloud), GraphQL cursor (GitHub and GitLab GraphQL), keyset (GitLab REST), and none (CLI based sources, AWS WAF sampled requests). The sources also split on an **operational pattern** axis. Periodic global scanners (SonarQube, Dependency-Track, Semgrep Cloud) are polled via `updated_at` cursors. CI/CD step scanners (Semgrep Docker, TruffleHog) run per commit and are indexed by commit SHA. On demand dynamic scanners (OWASP ZAP) are read via the scan lifecycle API. Runtime telemetry sources (AWS WAF) are sampled over time windows. Together these exercise all five ingestion strategies the framework prescribes.

The source capability matrix reference page tabulates the nine sources against the capabilities that matter for connector design. This matrix drives the derivations in Section 2.2: the Silver Entity and Silver Finding patterns must accommodate sources varying across every column, and the connector framework rules in Section 2.4 must commit to defaults that work uniformly.

This thesis fills the gap with a vendor agnostic framework for application security data. It applies lakehouse architecture and the medallion pattern to ingest tool output, normalize it into a domain model, and serve it through OLAP and OLTP interfaces. A new source needs only a connector module. Chapter 2 presents the framework, and Chapter 3 demonstrates its implementation.

2. Framework

Chapter 1 analyzed the sources, patterns, and gaps in enterprise application security data integration. This chapter presents the proposed framework: a reusable blueprint defining the architecture, data model, and patterns an organization follows to build its own implementation. The framework targets the Databricks Lakehouse, separating general patterns from platform specifics. Chapter 3 then demonstrates a concrete implementation.

2.1 Solution Architecture

This section presents the framework architecture at three levels: technology stack, component design, and data layer. The architecture addresses the requirements from domain analysis: heterogeneous source ingestion, vendor agnostic normalization, dual OLAP/OLTP data serving, and extensibility for new data sources and analytics.

A further design principle applies to the framework artifacts: every template defined in this chapter (schema patterns, mappings, connector contract) is declarative rather than procedural, so that an AI coding agent reading the specification has enough material to produce a candidate implementation. The published skill catalog, which sits on the external specification site and wraps these templates into executable prompts, is evaluated separately in Section 3.6.3.

2.1.1 Technology Stack

The framework targets the Databricks platform on AWS. Databricks separates storage from compute: data resides in object storage and compute scales independently. This reduces costs for uneven workloads typical for security data integration (Armbrust et al., 2021).

2.1.1.1 Delta Lake

Delta Lake is an open table format that adds ACID transactions, schema enforcement, and time travel to cloud object storage (Armbrust et al., 2020; Lee et al., 2024). It provides the storage layer for all three medallion tiers. Schema evolution lets bronze tables absorb new source fields without pipeline changes. Time travel enables auditing and rollback, critical for a security data platform where data integrity and reproducibility are essential. Armbrust et al. (2021) showed that lakehouse combines warehouse governance with data lake flexibility.

2.1.1.2 Unity Catalog

Unity Catalog provides unified data governance across the lakehouse (Databricks, 2025f). It manages a three level namespace (`catalog.schema.table`) that maps to the medallion layer organization: one catalog per environment, schemas for each source (bronze) and each domain (silver, gold). Fine grained access control enforces least privilege access to security data, which often contains sensitive vulnerability details. Automated lineage tracking records data flow from source through transformations to consumption, supporting auditability. Column level tags classify sensitive fields such as CVE descriptions and remediation guidance.

2.1.1.3 Lakeflow Declarative Pipelines

LDP is the pipeline framework for batch and streaming transformations (Databricks, 2025d). Pipelines are declared as Python or Structured Query Language (SQL) functions with explicit dependencies. The framework resolves execution order automatically. Data quality expectations are embedded directly in pipeline definitions as declarative constraints (e.g. “severity must be critical, high, medium, or low”). Records which violate expectations are quarantined and the pipeline keeps running. This follows the quarantine pattern from Section 1.6.2. LDP handles incremental processing natively through change data feed, reducing the cost of reprocessing for large datasets.

2.1.1.4 Lakebase

Lakebase is a serverless PostgreSQL database that provides the OLTP serving layer (Databricks, 2026b). It shares the underlying storage layer with the lakehouse, exposing gold layer tables as OLTP queryable without data duplication. This satisfies the dual serving requirement from Section 1.6.1: OLAP queries go through SQL warehouses for dashboards, and OLTP queries go through Lakebase for issue tracker integration, state lookups, and serving API. Lakebase scales to zero, reducing cost for workload spikes. Instant database branching creates isolated copies for development and testing without duplicating data.

2.1.1.5 Platform Synergies

Running all components on a single platform provides governance, lineage, and compute benefits that would require significant integration effort across disparate tools. Unity Catalog governs access uniformly across bronze, silver, and gold. The shared compute and storage model eliminates data movement between analytical and operational stores.

The platform also enables integration with Lakewatch, the Databricks SIEM analyzed in Section 1.7.2. Both systems share Delta Lake storage and Unity Catalog governance. The gold layer outputs of the framework (application risk scores, remediation status) can serve as enrichment signals for Lakewatch threat detection, while Lakewatch runtime security events could feed back as an additional finding source. This lines up with the finding in Section 1.7.2.

2.1.2 Component Design

The framework is organized into five tiers, illustrated in Figure 2.1. Data flows from top to bottom, i.e. from sources through the platform to consumers.

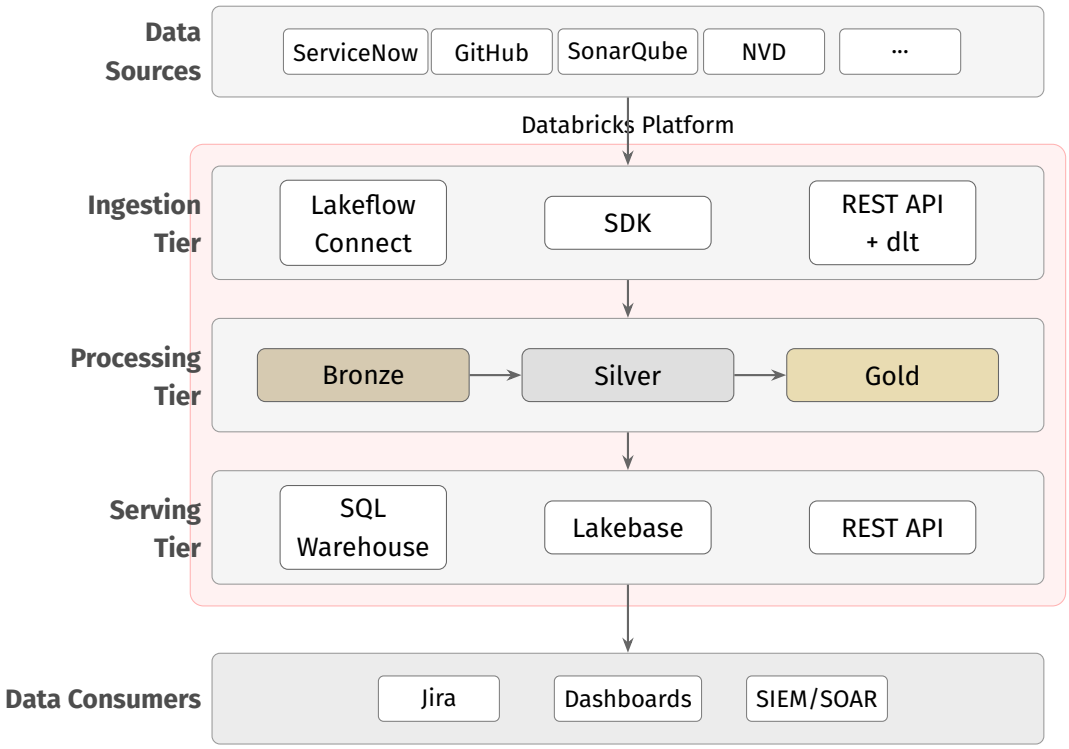

Figure 2.1
Component Design

Data sources are the external systems analyzed in Chapter 1: application inventories, SCM platforms, CI/CD tools, security scanners, and vulnerability databases. They are outside the framework boundary. The framework consumes their APIs but does not control them.

Ingestion tier contains connectors that extract data from sources and store it in the bronze layer. Three connector categories serve different integration scenarios. Lakeflow Connect uses a declarative managed ingestion pattern. Software Development Kit (SDK) connectors use source provided client libraries for programmatic extraction. REST API connectors use the open source dlt library for sources which lack Lakeflow Connect or dedicated

SDK. All connectors share common concerns: authentication, pagination, rate limiting, and incremental state. The connector framework in Section 2.4 defines these patterns.

Processing tier implements the medallion architecture (Section 2.1.3). Delta Lake provides the storage layer. LDP orchestrates the transformations. Bronze stores raw ingested data in source native schemas. Silver normalizes it into the vendor agnostic entity and finding model defined in Section 1.6.3. Gold computes aggregated metrics, ML enriched scores, and consumption ready datasets. Unity Catalog governs access and tracks lineage across all three layers.

Serving tier exposes processed data to downstream consumers. SQL warehouses serve OLAP queries for dashboards and analytics. Lakebase serves OLTP workloads: low latency lookups and operational APIs. A REST API layer provides HTTP access to the OLTP store.

Data consumers are external systems reading from the serving tier: issue trackers (Jira, ServiceNow), SIEM/SOAR platforms, and custom reporting tools. Like data sources, consumers are outside the framework boundary.

2.1.3 Medallion Architecture

The processing tier applies the medallion architecture pattern from Section 1.6.1. Each layer is a Delta Lake schema within a single Unity Catalog, providing unified governance and cross layer lineage. Figure 2.2 illustrates the layer structure and transformations between them.

2.1.3.1 Bronze Layer

The bronze layer stores raw data from each source with none or minimal transformation. Each connector writes to its own bronze schema (e.g., `github`, `servicenow`, `sonarqube`), preserving the native data structure. Records are appended with standard metadata columns: ingestion timestamp, source system identifier, and batch identifier. No business logic is applied. The goal is a precise, traceable copy of source data.

Bronze tables use schema-on-read. New fields from source API changes are accepted through incremental schema evolution without disrupting pipelines. Partitioning by ingestion date enables efficient time range queries. Records failing structural validation at ingestion (malformed JSON, missing required fields) are routed to quarantine tables per data source, along with diagnostic metadata.

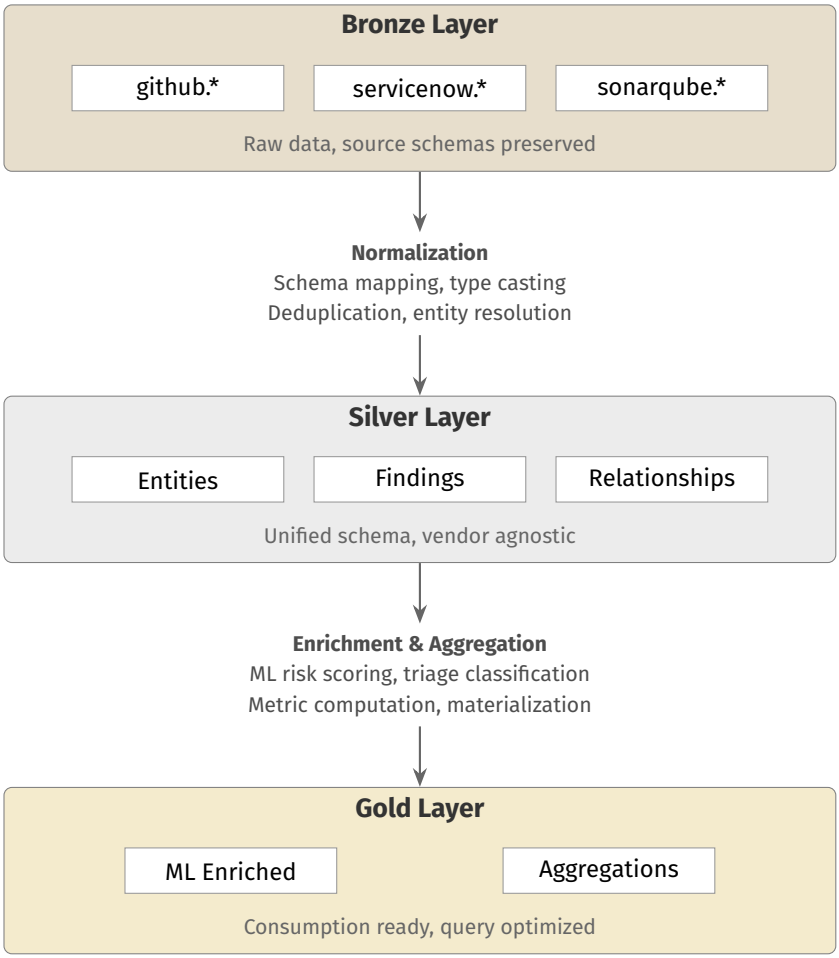

Figure 2.2
Medallion Layer Design

2.1.3.2 Silver Layer

The silver layer is the system of record. Silver transformations normalize diverse source data into the vendor agnostic domain model from Section 1.6.3. Three table categories make up this layer:

- **Entity tables** store normalized dimension data: applications, repositories, teams, commits, pull requests, pipeline runs, dependencies, and branch protection policies. Each entity table uses a surrogate key, a natural key from the source system, and `valid_from`/`valid_to` timestamps for change tracking.
- **Finding tables** store normalized security findings as fact records. Each finding carries a severity mapped to the standard scale, a lifecycle status, the source tool identifier, and references to the affected entity (repository, application). Finding tables are organized by category: SAST, SCA, secrets, DAST, containers, and IaC.
- **Relationship tables** store many to many mappings from application to repository, from finding to CVE, and cross tool deduplication links.

The silver layer applies the data quality patterns from Section 1.6.2: severity harmonization, entity normalization, timestamp standardization, and deduplication. LDP expectations enforce constraints declaratively. Records which violate expectations go to quarantine.

2.1.3.3 Gold Layer

The gold layer computes consumption ready datasets from silver data. Two categories of gold tables serve distinct purposes:

- **Aggregation tables** compute metrics at defined levels: application risk scores, team remediation rates, MTTR by severity, SLA compliance percentages, and time series trends. These tables power the dashboards and executive reports consumed through the OLAP serving path.
- **ML enriched tables** store model outputs: composite risk scores, false positive predictions, and remediation time estimates. These scores augment the aggregation tables with predictive signals. The ML workflow patterns are detailed in Section 2.5.1.

Gold tables use incremental refresh where possible: new silver records trigger recomputation of only the affected aggregations. Full refresh is meant for metrics requiring global recomputation, such as cross application percentiles.

2.2 Data Model

This section maps the conceptual domain model from Section 1.6.3 to physical table designs across the three medallion layers. Reusable schema patterns are applied to produce the concrete entity, finding, and aggregation tables that the framework provides.

2.2.1 Source Synthesis

The schema patterns are derived from the APIs of the nine sources introduced in Section 1.8, not chosen in advance. The per category capability matrix consolidates the observations relevant to schema design at the category level. The connectors reference hub provides facts for each source. This section maps the consolidated capabilities onto concrete Entity, Finding, and Relationship schemas.

2.2.2 Schema Patterns

Schema patterns are reusable templates that standardize how tables are created at each medallion layer. They ensure consistency and prepare the model for new entities or sources. Four patterns cover the framework needs. Figure 2.3 walks a single SonarQube SAST finding through the four layers that the subsections below define, and concretizes the metadata envelope that Section 2.2.2.1 prescribes.

2.2.2.1 Bronze pattern

Every bronze table carries a uniform metadata envelope: four columns shared across connectors (`_ingestion_timestamp`, `_source_system`, `_batch_id`, `_raw_payload`) and a fifth (`_hwm_value`) on incremental only tables (see Section 2.4.2). For connectors whose bronze schema is framework owned (artifact path sources such as OWASP ZAP and Semgrep), the bronze envelope helper stamps the envelope immediately before the write. For connectors whose bronze schema is owned by an ingestion managed service (Lakeflow Connect, as in the ServiceNow case), the envelope is published by a downstream SQL view that projects the five columns on top of the managed table using the ingestion service's own run identifier and timestamp metadata. The net contract is identical to downstream readers: every bronze row exposes the same envelope columns regardless of which of the three ingestion paths produced it.

Additional columns from the source native schema are included alongside the raw payload via schema on read. New fields are accepted through additive schema evolution. Tables are partitioned by ingestion date for retention policies and efficient time range queries.

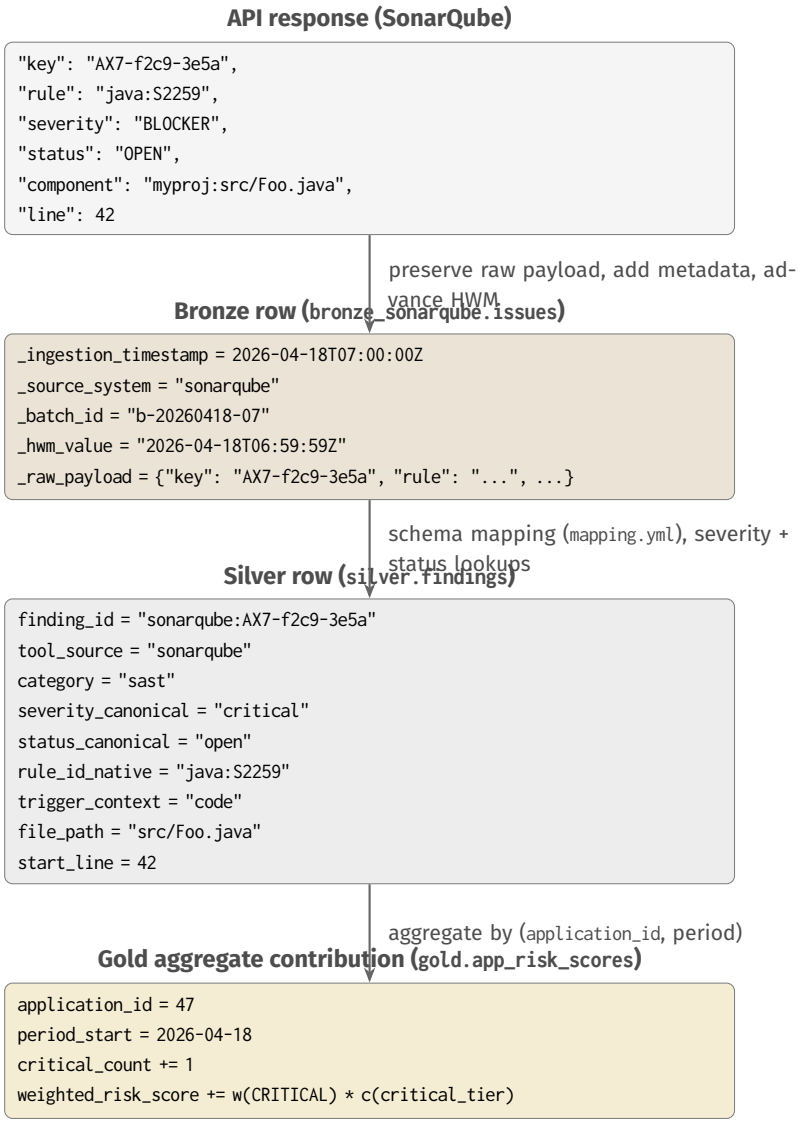

Figure 2.3

Lifecycle of a single SonarQube SAST finding from API response through Bronze ingestion with the metadata envelope with five columns, Silver normalization into `silver.findings` *with category discriminator, and Gold aggregation into* `app_risk_scores`*. The same logical record appears in each medallion layer with additional metadata and normalized fields added at each transition.*

2.2.2.2 Silver entity pattern

Entity tables store normalized dimension data. The pattern is derived from the entity emitting sources in the selection (ServiceNow CMDB records, SCM repositories and commits, team references). The field derivation table per source is published externally at https://vkraus.github.io/appsec-mvp/platform/reference/canonical-mapping/#silver-entity-mapping-requirements, showing how each target field unions over the native fields these sources expose.

The natural key plus source system uniquely identifies a record origin. The surrogate key provides a stable reference for foreign keys even when source identifiers change.

2.2.2.3 Quarantine scope

The transformation from Bronze to Silver for entity tables quarantines records under three conditions. First, records failing JSON parse at Bronze landing. Second, records failing required field null checks at the Bronze to Silver step (`natural_key`, `source_system`, `valid_from`). Third, records where a mandatory lookup (for example, severity or status for finding related joins) yields no match. Optional field nulls and type casts with safe fallbacks are accepted with a data quality warning logged to the pipeline event table but are not quarantined. This rule applies uniformly to finding tables in Section 2.2.2.

2.2.2.4 CMDB relationship resolution

The ServiceNow CMDB exposes inter-CI relationships through a separate `cmdb_rel_ci` table and reference fields on the CIs themselves. Related CMDB tables (`cmdb_rel_ci`, referenced group and user tables) are ingested as separate Bronze tables and joined in Silver rather than resolved at ingestion via relationship APIs. This keeps `ingest.py` stateless (it does not follow API links), makes relationship logic testable without API mocks, and localizes CMDB schema changes to the transformation layer.

2.2.2.5 Silver finding pattern

The single Silver Finding table (`silver.findings`, whose design rationale appears in Section 2.2.3.2) stores normalized fact data for all finding categories. The pattern is derived from the four finding emitting sources (SonarQube, Semgrep, Dependency-Track, TruffleHog) plus the scanners integrated into the platform on GitHub and GitLab. The derivation tables per source for code level and package/platform sources are published externally at https://vkraus.github.io/appsec-mvp/platform/reference/canonical-mapping/#silver-findin

g-mapping-requirements. Each target field unions over the native fields of these sources and inapplicable fields are marked "N/A".

Standard fields marked "N/A" for a source are stored as `NULL` in records from that source. This is the union over sources behavior. The `mapping.yml` per source (Section 2.4.3) makes each mapping explicit, including the `category` discriminator for each record.

2.2.2.6 Deduplication pattern

When multiple tools scan the same artifact, overlapping findings must be identified and linked rather than collapsed. Deduplication is applied within `silver.findings` using a category conditional exact match tuple selected by the `category` value of each record:

- SAST findings (`category = 'sast'`): `(repository_id, file_path, rule_id, line_number)`. A match across two SAST tools links the two findings via a `dedup_links` relationship record.
- SCA findings (`category = 'sca'`): `(repository_id, package_name, cve_id)`. Two SCA tools reporting the same CVE on the same package are duplicates.
- Secret findings (`category = 'secret'`): `(repository_id, commit_sha, secret_type, file_path)`. Secrets emitted per commit are deliberately retained per commit rather than collapsed by secret value.

No fuzzy matching is performed. If two tools diverge on line numbers (for example, due to file preamble differences), both findings are retained. They are grouped at Gold via a `finding_group_id` clustering findings sharing `(repository_id, file_path, rule_id)` independent of line number. Chapter 3 instantiates this logic verbatim, without additional heuristics.

2.2.2.7 Relationship pattern

Relationship tables store many to many mappings between entities. Each contains foreign keys to both sides, a source system indicator, and `valid_from`/`valid_to` timestamps tracking when the relationship was active. No payload columns are included. The sole purpose of the table is to link entities. Examples include mappings from application to repository, from finding to CVE, and cross tool deduplication links.

2.2.2.8 Gold aggregation pattern

Aggregation tables follow a grain, metric, period structure:

- **Grain columns** define the level of detail: the entity key (application, team, repository) and time period (day, week, month).

- **Metric columns** store computed values: finding counts by severity, MTTR, SLA compliance percentage, risk score.
- **Period columns** store the time window: `period_start` and `period_end` (Coordinated Universal Time (UTC)).
- **Refresh metadata**: `computed_at` timestamp and refresh strategy indicator (incremental or full).

This structure enables consistent querying across gold tables: filter by grain, aggregate over periods, compare across entities.

2.2.3 Entity Model

The silver layer instantiates the schema patterns as concrete tables, organized by category: entities (dimensions), findings (facts), reference data, and relationships.

Figure 2.4 presents a representative subset that exercises every relationship type across the four categories. The full table inventory follows in the subsections below.

2.2.3.1 Entity Tables

Table 2.1 lists the entity tables and their key domain specific columns. All tables include the standard silver entity pattern columns (`id`, `natural_key`, `source_system`, `valid_from`/`valid_to`).

2.2.3.2 Finding Table

All security findings land in a single Silver table, `silver.findings`, distinguished by a `category` discriminator column taking values `sast`, `sca`, `secret`, `dast`, `container`, or `iac`. Category specific attributes are carried as nullable columns, populated only for records where the category defines them. Table 2.2 lists the category specific columns. Every finding references its repository through `repository_id`. Business application context is derived through relationship tables rather than stored directly on findings, preserving the many to many mapping from application to repository without duplicating finding records.

2.2.3.2.1 Single table rationale

The Silver Finding mapping is already a union over sources. A physical split into `silver.sast_findings`, `silver.sca_findings`, and so on would force `UNION ALL` queries for cross category analytics, fragment the per category dedup routing keyed on `category`, and distort

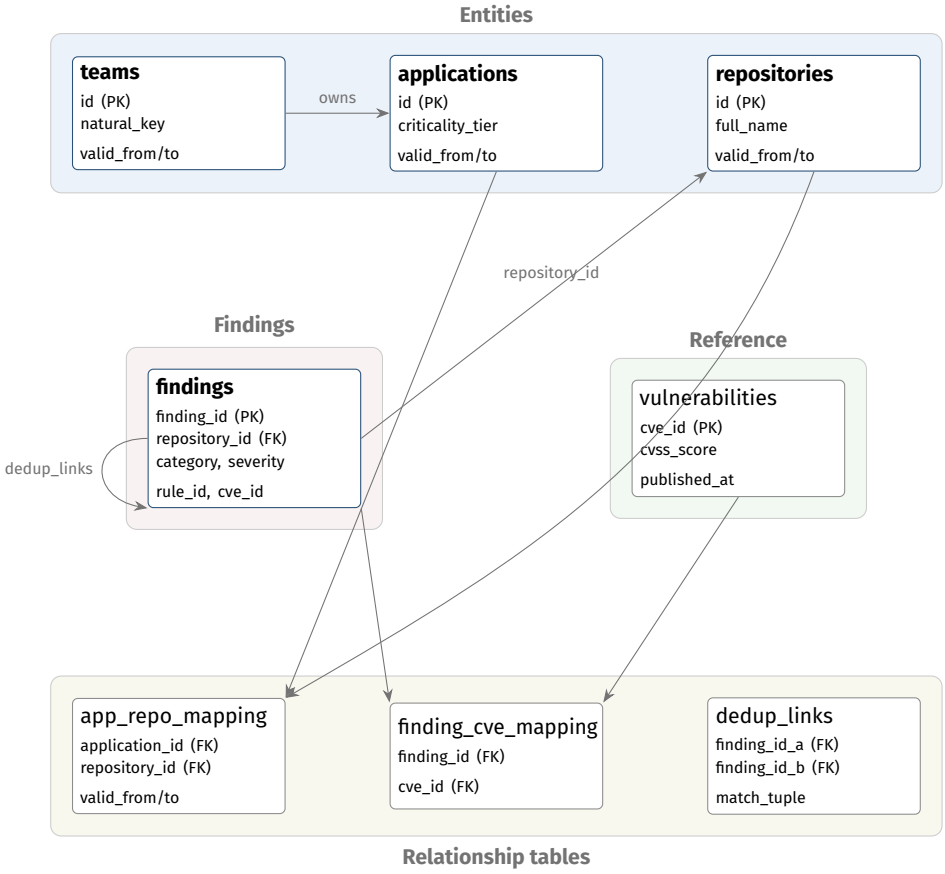

Figure 2.4

*Silver layer entity relationship diagram. Representative subset: **8** of the **15** silver tables. Four groupings are visible through background shading: entities, findings, reference, and relationship tables. The remaining 7 tables follow the same schema patterns.*

Table 2.1
Silver Entity Tables

Table	Source	Key Domain Columns
applications	CMDB	name, criticality_tier, lifecycle_status, business_unit, compliance_scope
repositories	SCM	full_name, primary_language, visibility, default_branch, archived, last_activity_at
teams	CMDB/SCM	name, parent_team_id, contact_email
commits	SCM	sha, author, authored_at, message, repository_id
pull_requests	SCM	number, title, state, author, created_at, merged_at, repository_id
pipeline_runs	CI/CD	pipeline_id, commit_sha, status, started_at, finished_at, repository_id
dependencies	SCA	package_name, installed_version, license, ecosystem, repository_id
branch_policies	SCM	required_reviewers, require_status_checks, enforce_admins, repository_id

Table 2.2
Category specific columns of `silver.findings`*. Each record populates the columns applicable to its* `category`*. Unused columns are* `NULL`*.*

Category	Category specific columns
sast	file_path, line_start, line_end, cwe_id, language
sca	package_name, installed_version, fixed_version, cve_id, ecosystem
secret	secret_type, file_path, commit_sha, validity_status
dast	url, http_method, parameter, attack_type
container	image_name, image_tag, layer, cve_id
iac	resource_type, resource_name, file_path, policy_id, benchmark

Note. Column list abstracted from the silver layer schema definition. The full specification is on the silver table ownership reference page.

the category partitioned requirement traceability matrix. Sparse nullable columns are the accepted cost. At the target deployment scale, Delta Lake's columnar compression makes NULLs cheap. The full design rationale is on the single table findings rationale page.

2.2.3.3 Reference Tables

Reference tables store external intelligence used for enrichment:

- **vulnerabilities**: CVE records with description, published date, and CVSS base score from the NVD.
- **epss_scores**: daily EPSS probabilities per CVE, enabling time series analysis of exploitation likelihood.
- **kev_entries**: CISA KEV catalog entries indicating confirmed active exploitation, with the date added.

These tables are refreshed on schedule from their external sources. Findings link to them through CVE identifiers, forming the three signal enrichment model described in Section 1.3.

2.2.3.4 Relationship Tables

Three relationship tables implement the key mappings identified in the domain model:

- **app_repo_mapping**: links applications to repositories (many to many). This is the most critical relationship for business context attribution.
- **finding_cve_mapping**: links findings to CVE records, enabling vulnerability enrichment.
- **dedup_links**: links findings identified as duplicates across tools, preserving traceability to each source report while establishing a reference finding.

Silver tables and connectors do not line up one to one. Each table can be fed by multiple connectors and each connector feeds multiple tables. The full mapping is on the silver table ownership reference page, which turns new source onboarding into a checklist of target tables.

2.2.3.5 Reference Implementation Scope

The fifteen table inventory above defines the full silver layer of the framework. The reference implementation in Chapter 3 realizes a named subset sufficient to demonstrate every framework concept end to end without carrying all fifteen tables into production: `applications` and `repositories` as entity tables, `findings` as the fact table, and `app_repo_mapping` as the critical relationship. The remaining entity tables (`teams`, `commits`, `pull_requests`, `pipeline_runs`, `dependencies`, `branch_policies`), the reference tables (`vulnerabilities`, `epss_scores`, `kev_entries`), and

the derived relationship tables (`finding_cve_mapping`, `dedup_links`) follow the same schema patterns and are left to follow on work in Section 3.6.3. The reference implementation also realizes the silver entity columns as Slowly Changing Dimension (SCD)-1 (overwrite on update) rather than the SCD-2 pattern (`valid_from`/`valid_to`) documented for the framework, since the MVP scenarios do not exercise historical queries at a point in time.

2.2.4 Aggregation Model

The gold layer computes consumption ready metrics from silver data using the aggregation pattern. Each gold table targets a specific stakeholder need identified in Section 1.6.3.

2.2.4.1 Application risk scores

The `app_risk_scores` table computes a composite risk metric per application per period. Grain columns are `application_id` and time period. Metrics include: open finding counts by severity, weighted risk score (combining severity, EPSS probability, and application criticality tier), SLA compliance percentage, and a trend indicator (improving, stable, degrading). It serves application owners who need a single view of the security posture for their application.

2.2.4.2 Team metrics

The `team_metrics` table aggregates remediation performance per team per period. Metrics include: MTTR by severity, finding closure rate, new vs. resolved finding ratio, and SLA breach count. Leadership uses these metrics for cross team comparisons and resource allocation decisions.

2.2.4.3 Vulnerability trends

The `vulnerability_trends` table provides time series data at configurable intervals (daily, weekly, monthly). Metrics include: new findings introduced, findings resolved, net open count, severity distribution shifts, and mean age of open findings. These trends power the longitudinal dashboards that track organizational progress.

2.2.4.4 Coverage analysis

The `coverage_analysis` table identifies gaps in security tool coverage. For each repository, it records which tool categories have produced findings or scan records and which have not. Missing coverage is flagged per category: a repository with no SAST findings and no

SAST pipeline runs likely lacks static analysis integration. This lets security teams prioritize tooling rollout.

2.2.4.5 Extension blueprint

Adding a new gold table follows a consistent process: define the granularity (entity and time period), specify the metrics, write a LDP transformation from silver to gold, and configure the refresh strategy. The aggregation pattern ensures all gold tables share a common query interface, so dashboards and reporting tools can consume new tables without structural changes.

2.3 Environment and Deployment

Before the components and data structures defined above can be populated with connectors or analytics, the deployment environment must be provisioned and the codebase organized. **This section assumes a working Databricks environment.** Account onboarding, workspace creation, networking, and Identity and Access Management (IAM) federation are outside the scope. The scope of the framework begins at the workspace level and divides its own deployment into two tiers: workspace scoped resources are provisioned with Terraform, and application artifacts run inside the workspace as Databricks Asset Bundles. The rest of this section covers those two tiers, the project structure they organize, pipeline orchestration, monitoring, and deployment level verification. These are one time decisions that establish the platform. Repeatable patterns for extending the framework follow in Section 2.4 and Section 2.5.

2.3.1 Deployment Strategy

The framework deploys in two tiers. The **workspace tier** provisions workspace scoped resources with Terraform via the Databricks Terraform provider (Databricks, 2025b): catalog creation (one per environment, per Section 2.1.1), cluster policies, secret scopes for the source credentials in Section 1.8, service principal identities, and coarse permission grants. A single `workspace-bootstrap/` module captures the full set, so one `terraform apply` recreates the workspace state for a new environment or disaster recovery target. The **application tier** deploys everything inside the provisioned workspace. It uses Databricks Asset Bundles (Databricks, 2025e) as the deployment unit: a single `databricks.yml` at the project root declares jobs, LDP pipelines, notebooks, permissions, and cluster attachments together with the source code that implements them.

One bundle exists per target. Three targets structure the promotion path: development, staging, and production. Each target overrides the bundle parameters that differ between

environments: the Unity Catalog catalog name supplied by the workspace tier, the secret scope that resolves API credentials, and the compute target for jobs and pipelines. Development uses smaller clusters and relaxed permissions for rapid iteration. Staging mirrors production for prerelease validation. Production enforces strict access controls and scheduled execution. Because both the workspace module and the bundle are declarative, the same promotion flow moves a change through the three environments without hand editing any resource after the first deployment. The framework prescribes only this two tier boundary. Organizations that prefer Pulumi, OpenTofu, or direct Databricks REST API calls can substitute those at the workspace tier without changing the application tier. DAB and the Databricks Terraform provider are selected for the reference implementation because they are Databricks native and integrate with the audit and lineage story of the platform.

2.3.2 Project Structure

The project follows a monorepo layout with four top level directories: `src/` for pipeline source code, `tests/` for the test suite, `config/` for lookup tables, and the bundle files at the root. The Unity Catalog layout mirrors the three medallion layers from Section 2.1.3, with one catalog per environment as established in Section 2.3.1: `appsec_dev`, `appsec_staging`, and `appsec_prod`. Within a catalog, source specific bronze and silver tables live in schemas per source named `bronze_<source>` and `silver_<source>` (for example, `bronze_github` and `silver_servicenow`). Cross source silver tables (finding aggregation, cross entity joins, framework control tables) live in an unqualified `silver` schema, and gold analytics live in an unqualified `gold` schema whose table names encode the analytic rather than a source. A production ServiceNow business applications row therefore lives at `appsec_prod.silver_servicenow.business_applications`. The cross source finding aggregation lives at `appsec_prod.silver.findings`. The app risk score analytic lives at `appsec_prod.gold.app_risk_scores`. Pipeline names mirror their source module as `ingest_<source>` and `transform_<source>_silver`. Column names use `snake_case` throughout. These conventions let governance rules target tables and pipelines by name pattern rather than by enumeration.

Every connector module under `src/connectors/{source}/` carries the same four artifacts, so adding a source amounts to filling in the blanks. `ingest.py` implements the connector contract from Section 2.4.1 against the source API. `transform.py` maps bronze records to the target silver entity or finding table. `mapping.yml` declares the column expressions from bronze to silver and references the severity and status lookups from Section 2.4.3. `config.yml` records source specific parameters: base URL and endpoints, pagination strategy, high water mark column, and target bronze table. A colocated `tests/connectors/{source}/` subfolder carries the fixtures and assertions from Section 2.4.4. Transformations from silver to gold are grouped by analytic rather than by source, since a single gold table can consume data from multiple connectors.

Configuration is split by sensitivity. Secrets such as API tokens and service account credentials are stored in the platform secret scope and referenced by name in pipeline code,

so they never appear in source files or bundle configuration. Nonsensitive settings sit under `config/`: severity lookups at `config/severity/{source}.yml` and status lookups at `config/status/{source}.yml`. SLA thresholds and scheduling intervals are out of scope for the MVP configuration. Tuning any of these values is a configuration change, not a code change. The full layout with a worked example is on the project layout reference page.

2.3.3 Pipeline Orchestration

Lakeflow Jobs schedules and coordinates pipeline execution (Databricks, 2026c). A job groups related tasks into a Directed Acyclic Graph (DAG) with explicit dependencies. The engine executes tasks in dependency order and handles retries on failure. Jobs are defined in the bundle configuration alongside the resources they operate on, making scheduling version controlled and promotable via the deployment path from Section 2.3.1.

The framework assigns one job per connector. Each connector job contains two sequential tasks: an ingestion task that extracts data from the source API into bronze, and a transformation task that normalizes bronze records into silver. This isolation gives each connector an independent failure domain: a GitHub API outage does not block ServiceNow ingestion. Gold layer analytics run as separate jobs listing their upstream connector jobs as dependencies. The scheduler starts a gold job only after all required silver data is fresh. ML model retraining runs on independent schedules, decoupled from the ingestion cycle.

Source characteristics drive scheduling frequency. High change sources (SCM platforms, security scanners with frequent new findings) run hourly. Stable sources (application inventory from the CMDB) run daily. External enrichment sources follow their update cadences: NVD and EPSS update daily, CISA KEV updates on weekdays. Gold analytics trigger after upstream connector jobs complete via explicit dependencies. Where strict ordering adds unnecessary complexity (e.g., daily reporting), a schedule offset from the expected ingestion window is acceptable.

Each task has a retry count and backoff interval for transient failures. If retries exhaust, the task fails and downstream tasks in the same job do not execute. Job level alerts notify operators of persistent failures through configured channels (email, messaging webhooks). Since connector jobs are independent, a single connector degradation does not cascade. Jobs run in parallel on ephemeral clusters created at job start and terminated at completion.

2.3.4 Monitoring and Observability

Scheduling handles when pipelines run. Monitoring handles whether they are working over time. The platform provides system tables that record job execution history, pipeline events, and compute utilization (Databricks, 2026d). The framework builds on these to track three dimensions. **Data freshness** compares the last successful ingestion timestamp on each

bronze table (and last transformation run on each silver table) against thresholds derived from the scheduling frequency in Section 2.3.3. An hourly connector idle for two hours or a daily enrichment source idle for 36 hours flags as stale. **Pipeline health** aggregates job execution metrics: success rate over a rolling window, mean run duration and its trend, task level retry frequency, and quarantine volume. A rising quarantine rate precedes data quality issues in silver. **Data quality trends** track violations of LDP expectations over time. Alerts fire at two levels. Job level alerts cover individual failures or threshold breaches. System level alerts cover aggregate degradation such as multiple connectors stale simultaneously or gold tables not refreshed within their expected window. Thresholds and delivery channels (email, messaging webhooks, incident management integrations) are defined as configuration, so operators tune sensitivity without redeploying pipelines.

2.3.5 Testing and Validation

Deployment level verification covers the environment and codebase. Component level patterns live in Section 2.4.4 and Section 2.5.3. Before deployment, the CI/CD pipeline enforces linting, formatting, and unit tests over isolated helpers such as severity mapping lookups, timestamp parsers, and schema mapping logic. After deployment, smoke tests confirm Unity Catalog objects, compute resources, secret scopes, and LDP pipelines exist with the expected configuration. Failures block pipeline execution until corrected.

The framework uses pytest markers to link tests to requirement identifiers. Each test function carries a marker referencing a specific requirement from the requirement catalog, such as `@pytest.mark.requirement("REQ-TRF-SEV")`. A traceability matrix is generated automatically from test results and published at https://vkraus.github.io/appsec-mvp/platform/reference/catalog/#per-source-traceability-matrix, applying uniformly across environment, connector (Section 2.4.4), and analytics (Section 2.5.3) testing.

Four templates complete the framework. Environment and deployment (above). Data model (Section 2.2). Connector contract and job template (Section 2.4). Analytics blueprint (Section 2.5). Chapter 3 instantiates each verbatim. The reference implementation contains no design decisions beyond selecting which sources to onboard and which analytics to build.

2.4 Connector Framework

The connector framework defines how data moves from the sources analyzed in Chapter 1 into the medallion layers from Section 2.1.3. Section 2.1.2 introduced three connector categories (Lakeflow Connect, SDK, and REST API with dlt). This section specifies the common abstraction they share, the patterns for landing data in bronze, and the patterns

for transforming it to silver. The goal is a repeatable recipe: adding a new source implements a well defined set of concerns against a new API, not a pipeline redesign.

Figure 2.5 gives the end to end view refined by the subsections below: ingest and transform tasks along the spine, quarantine branches where validation can fail, and the state and dedup annotations that the framework commits to.

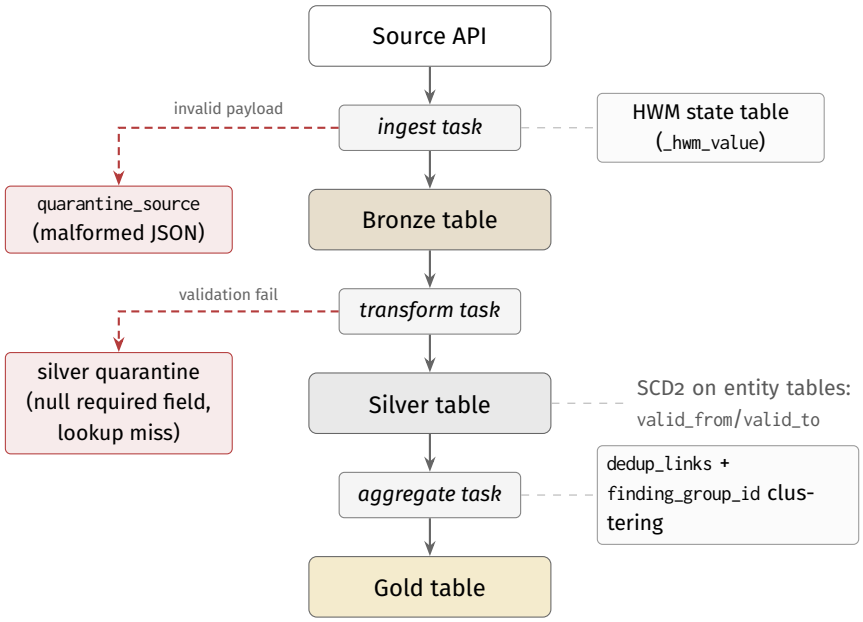

Figure 2.5
Data plane flow within the framework: ingestion with auditable high water mark state (right), quarantine on the Bronze side and on the Silver side for records that fail structural or required field validation (left), SCD2 tracking on entity tables, and materialization from Silver to Gold with cross tool deduplication.

2.4.1 Connector Abstraction

Every connector handles the same cross cutting concerns, but the three categories from Section 2.1.2 differ in how much work the developer performs versus the platform or library.

2.4.1.1 Connector Categories

Lakeflow Connect connectors use the managed ingestion service of the platform. They are declarative: configured through the bundle definition, not coded. Authentication, pagination, incremental state, and schema inference are handled by the platform. This suits sources with supported Lakeflow Connect integrations where full table ingestion meets requirements. The trade off is limited flexibility: custom transformation logic, selective field extraction, and nonstandard pagination cannot be injected. It ranks first in the preference order because no connector code is written for any of the five cross cutting concerns defined

below. The concerns migrate off the connector. The connector becomes a declaration that cannot drift from the framework contract.

SDK connectors use a source provided client library (e.g., the AWS SDK for Python, PyGitHub, python-gitlab) for programmatic extraction. These SDKs encapsulate authentication, pagination, rate limiting, and object model mapping, letting the developer work with high level methods rather than raw HTTP requests. This suits sources that offer an official SDK with good endpoint coverage. The developer controls extraction logic. The SDK handles transport level concerns. It ranks second because when Lakeflow Connect does not apply, the vendor SDK is the remaining option that keeps transport, pagination, and rate limit logic in library code maintained by parties closer to the source than the connector author. It is typed against the source object model, so upstream API changes appear as library upgrades rather than silent breakage in a hand written binding.

REST API connectors with dlt use the open source dlt library to build ingestion pipelines against sources lacking a dedicated SDK. The library provides prebuilt authentication, pagination, and incremental loading components composed declaratively against the source REST API. It handles mechanical concerns (request construction, response parsing, state management) while the developer specifies endpoints, schema mapping, and extraction parameters. It ranks third because it delegates the mechanical concerns to tested library components rather than hand written HTTP code. That is why it is the last sanctioned category, not a custom client. The developer must now identify endpoints, describe the response structure, and configure incremental loading. More connector code is written. More connector review is required.

The ordering works cumulatively. Each step moves one or more of the five concerns from platform custody into code per connector. Those concerns are auth, pagination, retry and back off, incremental state, and schema drift. More concerns in connector code means more places for defects and more code to track on upgrades. The preference order follows directly: use Lakeflow Connect if a supported integration exists and meets requirements, use the SDK for the source when one covers the needed endpoints, and fall back to dlt for sources with only a REST API. HTTP clients written by hand sit outside the order entirely and are not a sanctioned category, because they reintroduce every concern the three categories above delegate. The five responsibilities defined below form the full contract. REST API connectors implement them through dlt components. SDK connectors delegate them to the client library. Lakeflow Connect connectors satisfy them through platform configuration. Chapter 3 demonstrates all three categories.

Figure 2.6 summarizes the selection.

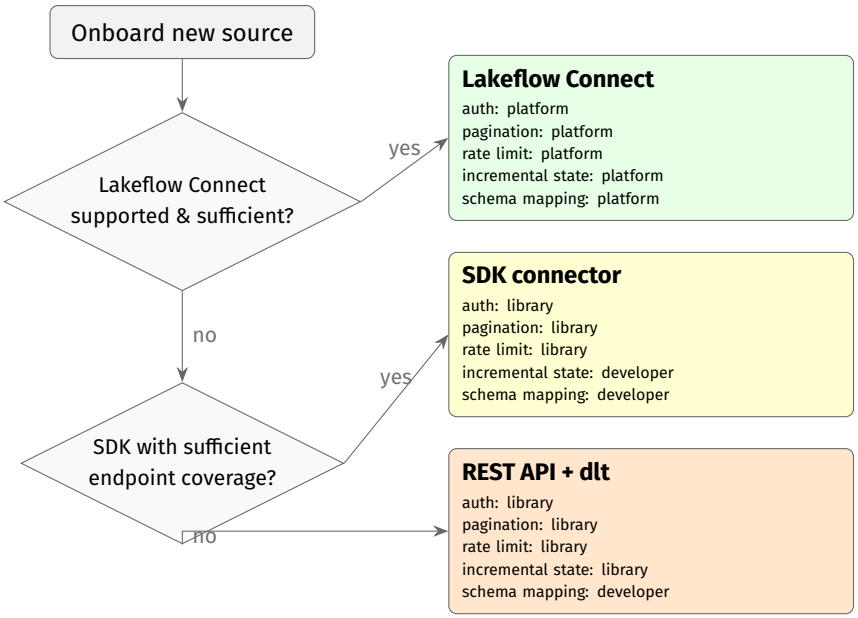

Figure 2.6
*Connector category selection. Terminal panels show how the five cross cutting concerns are handled: **platform** (managed), **library** (SDK or dlt), or **developer** (code per connector).*

2.4.1.2 Cross cutting concerns

2.4.1.2.1 Authentication

Security tool APIs use several authentication mechanisms: PATs, OAuth 2.0 client credentials, and service account keys. Credentials are externalized through a platform secret scope. Each connector declares which credential type it requires. The framework resolves it at runtime. This keeps secrets out of pipeline code and enables credential rotation without redeploying connectors.

2.4.1.2.2 Pagination

Section 1.5.2 shows REST APIs are the dominant integration style. These return results in pages. Two strategies cover most sources: offset based (page number and size parameters) and cursor based (an opaque token returned with each response). The connector abstraction hides pagination. Downstream code gets a stream of records. GraphQL based sources such as the GitHub API use cursor based pagination exclusively.

2.4.1.2.3 Rate limiting

Source APIs enforce rate limits, typically expressed as requests per time window. The connector implements adaptive backoff: on HTTP 429, it pauses for the duration in the response headers before retrying. For sources without rate limit headers, configurable rate

limits per source prevent exceeding undocumented thresholds. Transient errors (HTTP 5xx) trigger exponential backoff with a configurable maximum retry count.

2.4.1.2.4 Incremental state

Full re-ingestion does not scale at enterprise volumes (Section 1.6.2). Each connector maintains a high water mark: the timestamp or cursor of the last successfully ingested record. Subsequent runs resume from this position, fetching only new or changed records. The high water mark is persisted in a state table within the bronze schema, colocated with the data it tracks. For sources supporting CDC (e.g., a CMDB's update set mechanism), the connector consumes change events directly.

Two operational patterns drive the choice of high water mark type. **Periodic global** sources (server based scanners, CMDB platforms, SCM platforms polling `updated_at`) use a timestamp high water mark: the maximum `updated_at` observed in the last run. **CI/CD step** sources (scanners invoked per commit or per pull request, typically as containerized CLI tools) use a per repository commit SHA or run identifier: the connector records the last ingested commit per repository in the state table, and on the next run processes only scan artifacts whose commit SHA is newer. Both patterns use the same state table schema. They differ only in the semantics of the high water mark value.

2.4.1.3 Connector Contract

Every connector module exposes two entry points. `ingest` accepts a run identifier and a state object, pulls new records from the source API, and returns a batch descriptor listing the rows landed and the advanced high water mark value. `transform` accepts a bronze dataframe matching the schema of the connector and returns a silver dataframe conforming to the entity or finding pattern (Section 2.2.2). Neither writes to silver itself. The pipeline runner persists the returned dataframe. This keeps the connector free of storage concerns and makes both entry points testable with in memory fixtures. For artifact path connectors whose source specific ingestion primitive already occupies the `ingest` symbol (OWASP ZAP, Semgrep CLI), the contract wrapper is exposed as `ingest_contract`. Both names resolve to the same framework behaviour.

The state object has a fixed structure across categories: the high water mark value (timestamp or cursor) last committed by a successful run, the source system identifier, and optional backfill parameters for bounded replays. The runner loads state from the state table per source before `ingest` and persists the returned state after a successful write, giving consistent resume semantics regardless of whether the connector is built on Lakeflow Connect, a source SDK, or dlt. Lakeflow Connect connectors satisfy the contract through platform configuration. SDK and REST-API connectors implement the two operations directly and delegate cross cutting concerns to the corresponding library or helper.

2.4.1.4 Connector Artifacts

Adding a connector produces a set of artifacts that instantiate the contract and populate the module template from Section 2.3.2. The list is identical for all three categories. Differences are only in how individual artifacts are populated:

1. `src/connectors/{source}/config.yml`: source parameters, like the base URL, endpoints, pagination strategy (offset or cursor), rate limit, the high water mark column name, and the target bronze table name.
2. A secret scope entry (platform level, not a file) registering the credential the connector requires. The entry name is referenced from `config.yml`. Credentials never appear in source files.
3. `src/connectors/{source}/ingest.py`: implementation of `ingest(run_id, state) -> batch`. Lakeflow Connect connectors leave this file empty and declare the ingestion resource in the bundle fragment instead. SDK connectors delegate transport concerns to the client library. dlt connectors compose prebuilt authentication and pagination components.
4. `src/connectors/{source}/transform.py`: implementation of `transform(bronze_df) -> silver_df` targeting the silver table owned by the connector category (Section 2.2.3).
5. `src/connectors/{source}/mapping.yml`: declarative column expressions from bronze to silver consumed by `transform.py`, referencing the severity and status lookups below.
6. `config/severity/{source}.yml` and `config/status/{source}.yml`: severity and status lookups per source, maintained as configuration rather than code so that vocabulary updates do not require a pipeline redeploy.
7. A bundle fragment under `resources/` defining the connector job in the standard two task layout specified in Section 2.4.2.
8. `tests/connectors/{source}/`: ingestion and transformation tests with the fixture layout specified in Section 2.4.4.

Silver and gold do not change. The silver tables from Section 2.2.3 take the new source through the schema mapping. Gold analytics keep working.

2.4.2 Ingestion Patterns

Ingestion moves data from source APIs into bronze. All connectors follow the same landing pattern, with source type specific variations below.

2.4.2.1 Common landing pattern

Every ingestion run produces append only writes to the target bronze table. The full API response is preserved in `_raw_payload` alongside the metadata columns from the bronze

schema pattern (Section 2.2.2): ingestion timestamp, source system identifier, and batch identifier. No field filtering or transformation occurs. Bronze is an exact copy of source data.

The batch identifier enables idempotent replays. If a run fails partway through, reexecuting the same batch overwrites only the records of that batch, preventing duplicates. For sources where merge semantics are appropriate (e.g., CMDB entity snapshots), the connector uses upsert writes keyed on the natural identifier of each source record.

Records failing structural validation at landing, such as malformed JSON or responses with unexpected schema changes, are routed to quarantine tables per source. Each quarantine record includes the raw payload, error description, and batch identifier. No record is silently dropped. Every retrieved record either lands or goes to quarantine with a failure reason (DAMA International, 2017; Reis & Housley, 2022).

2.4.2.2 Bronze table template

Every bronze table instantiates the same standard structure extending the schema pattern from Section 2.2.2. The incremental only `_hwm_value` column introduced in Section 2.2.2 carries the high water mark value of the connector for the batch. This makes the resume position auditable per record and supports targeted rebuilds that replay ingestion from a chosen point without rereading the state table. Tables are partitioned by ingestion date to bound scan cost and enforce retention policies at the partition level. The naming convention `appsec_<env>.bronze_<source>.<entity>` aligns with the Unity Catalog layout from Section 2.3.2: for example, `appsec_prod.bronze_github.code_scanning_alerts` or `appsec_prod.bronze_servicenow.cmdb_ci_application`. A connector author fills in three blanks per entity: source name, entity name, and high water mark source field.

2.4.2.3 Connector job template

Every connector instantiates the same Lakeflow Job layout from Section 2.3.3: a two task DAG where an ingest task produces bronze records and a transform task consumes them to produce silver. The transform task declares a hard dependency on the ingest task, so a failed ingest short circuits the job without leaving silver partially refreshed. Retry configuration is identical across connectors (three attempts, capped exponential backoff), isolating transient source faults from pipeline faults. The bundle fragment exposes a fixed set of parameters: source name (used throughout resource naming), target catalog (environment scoped), a high water mark reset flag for manual backfills, and the schedule cron expression. The full bundle fragment is on the connector job template reference page. Each new connector copies it verbatim and substitutes the source name and credential reference.

2.4.2.4 Category specific considerations

Ingestion details by AppSec category are on the per category capability matrix. Incremental strategy preference order for SCM. Three scanner deployment styles (server based, CLI based, integrated into the platform). The CMDB relationship traversal rule. The common landing pattern and bronze table template above apply uniformly across all categories. Parameters per connector live in `src/connectors/{source}/config.yml`.

2.4.3 Transformation Patterns

Transformations move data from bronze to the silver entity and finding tables from Section 2.2.3. Each transformation is a LDP pipeline reading from one or more bronze tables and writing to a silver table. Five patterns compose the path from bronze to silver.

2.4.3.1 Schema mapping

Each connector has a schema mapping extracting typed columns from the bronze raw payload. The mapping defines which JSON fields correspond to which silver columns, applies type casts (e.g., string timestamps to UTC datetime), and assigns the surrogate key. Mappings are declared as column expressions in the LDP pipeline, making them easy to review and update when source schemas change.

The `mapping.yml` for every connector has the same structure: a target table declaration and a list of named column blocks where each block names a target Silver column, the JSON path into `_raw_payload`, and a type cast. Severity and status lookups live separately at `config/severity/{source}.yml` and `config/status/{source}.yml` as key-value maps from source to target, so vocabulary updates do not require a pipeline redeploy. A worked example for one source (SonarQube) is published at `https://vkraus.github.io/appsec-mvp/connectors/sast/sonarqube/#mapping-example`.

2.4.3.2 Normalization

Three normalization rules bring diverse source data to a common form:

- **Severity harmonization.** Tools use different severity scales, catalogued in Section 1.8 and detailed per source on the connectors reference hub. The framework maps the native scale of each tool to the standard four level model (critical, high, medium, low) through per tool lookup tables. The `severity_lookup.yml` for each source must map every documented source value to a canonical level. Undocumented values fall through to a configurable default (`medium` unless overridden in the `config.yml` for the

connector) and trigger a data quality warning. A null or missing severity is mapped to `medium` and flagged. These lookup tables are maintained as configuration, not code, enabling vocabulary updates without pipeline changes.

- **Status normalization.** Finding lifecycle states vary across tools (e.g., SonarQube's "confirmed" vs. GitHub's "open"). A per tool status mapping translates the states of each tool to the standard five state model (open, confirmed, resolved, false_positive, wontfix).
- **Timestamp standardization.** All timestamps are converted to UTC. Source specific formats (ISO 8601, Unix epoch, tool specific strings) are parsed during schema mapping and stored as UTC datetime columns.

2.4.3.3 Data quality validation

LDP expectations enforce constraints on every record entering silver. Expectations are declared alongside the transformation and checked at runtime. Examples: severity must be one of four standard values, status must be one of five standard states, repository foreign keys must reference an existing silver entity, and timestamps must fall within a plausible range. Violating records are quarantined rather than propagated, consistent with the quarantine pattern at ingestion.

2.4.3.4 Deduplication application

Deduplication keys are defined in the Silver Finding pattern (Section 2.2.2): exact match on `(repository_id, file_path, rule_id, line_number)` for SAST, on `(repository_id, package_name, cve_id)` for SCA, and on `(repository_id, commit_sha, secret_type, file_path)` for secrets. The transformation step applies these keys: every silver finding enters a post mapping dedup pass that writes `dedup_links` records for every overlapping pair among records sharing a `repository_id`. Deduplicated findings are not deleted. The link record preserves traceability to each source report while establishing the reference finding for aggregation. No fuzzy matching is performed. Line number divergence is handled at Gold via `finding_group_id` (Section 2.2.2). Deduplication pairs are enumerated per tool combination in Chapter 3.

2.4.3.5 Business context attribution

The final transformation step links findings to business applications. The `app_repo_mapping` relationship table maps repositories to applications. Since this is many to many (one application may span multiple repositories, and one repository may serve multiple applications), findings inherit application context through a join rather than a direct foreign key. This enables the gold layer aggregations in Section 2.2.4 to compute per application metrics.

The mapping is sourced from the CMDB connector and can be supplemented by automated inference from repository to application in the ML workflows from Section 2.5.1.

2.4.4 Testing and Validation

Connector tests live beside the connector they cover, at `tests/connectors/{source}/`, not in a central suite. `test_ingest.py` covers the ingestion contract, `test_transform.py` covers the transformation from bronze to silver, and `fixtures/` holds the JSON inputs both consume. Fixture filenames follow `{endpoint}_{scenario}.json` so regression cases are discoverable without opening each file. Fixtures are recorded once from a representative source instance and then frozen, which removes dependence on any particular tenant or tool build. The same test suite runs against any deployment, in CI/CD without network access, and a fixture change becomes an auditable diff rather than a silent environmental drift. The stack adds three libraries on top of pytest: an HTTP mocking library (`responses` or `requests-mock`), a PySpark DataFrame assertion library (`chispa`), and LDP expectations for runtime data quality checks. Each test carries a `@pytest.mark.requirement(...)` marker so the traceability matrix links results to the connector framework requirements below.

- `REQ-ING-AUTH`: authentication and credential resolution against the platform secret scope.
- `REQ-ING-PAG`: pagination traversal without data loss or duplication across at least two pages.
- `REQ-ING-RL`: HTTP 429 handling and backoff according to the configured retry policy.
- `REQ-ING-HWM`: high water mark resume across two consecutive runs with a midrun data change.
- `REQ-TRF-MAP`: schema mapping for every ingested endpoint.
- `REQ-TRF-SEV`: severity normalization covering every source specific severity value, including edge cases.
- `REQ-TRF-STS`: status normalization covering every source specific lifecycle state.
- `REQ-TRF-TS`: timestamp normalization for every source specific format.
- `REQ-DQ`: at least one LDP expectation per target silver table, with one violating and one valid record.
- `REQ-DEDUP`: deduplication for every applicable tool overlap pair (omitted when the connector category does not participate in dedup).

New connectors do not merge until every applicable marker above passes in CI/CD.

2.5 Analytics and Serving Framework

This section addresses the final stage: computing consumption ready insights from silver data and delivering them to stakeholders. Section 2.2.4 defined **which** gold tables the

framework provides. This section defines **how** those computations are structured and how results reach consumers through analytical, operational, and event driven channels.

2.5.1 Analytics Patterns

Gold layer computations fall into two categories: rule based analytics applying deterministic formulas to silver data, and ML driven analytics learning patterns from historical data. Both produce outputs following the gold aggregation pattern from Section 2.2.2.

2.5.1.1 Extension blueprint

Adding a new analytics workflow follows a consistent process. First, define the business question and identify the stakeholder. Second, choose rule based or ML: rule based suits well understood relationships with clear formulas, and ML suits pattern recognition over complex, high dimensional data. Third, implement the computation as a LDP pipeline (rule based) or MLflow tracked experiment (ML). Fourth, wire the output to a gold table following the aggregation pattern, configure the refresh strategy, and register the table in Unity Catalog for governance and lineage.

The process produces a fixed set of artifacts under `src/analytics/{name}/`, mirroring the connector layout from Section 2.3.2:

1. `src/analytics/{name}/pipeline.py`: the LDP pipeline (rule based) or MLflow experiment entry point (ML) that produces the gold table.
2. `src/analytics/{name}/ddl.sql`: the gold table schema declaration, conforming to the grain, metric, period structure from Section 2.2.2.
3. `src/analytics/{name}/config.yml`: refresh strategy (incremental or full), the aggregation grain, and the upstream silver dependencies.
4. A bundle fragment under `resources/` that wires the pipeline into the orchestration described in Section 2.3.3, declaring dependencies on the connector jobs that populate the upstream silver tables.
5. `tests/analytics/{name}/`: gold layer computation tests with known silver inputs and expected gold outputs. ML workflows add model training, validation, and drift tests per Section 2.5.3.

Authoring a new analytic never requires connector edits, and introducing a connector never requires analytics edits: silver is the stable boundary between the two.

2.5.1.2 Rule Based Analytics

Rule based analytics implement deterministic business logic as SQL transformations or LDP pipelines. Three computation patterns cover what the framework needs.

Composite scoring combines multiple signals into a single metric. The application risk score, for example, weights open finding counts by severity, multiplies by EPSS exploitation probability, and factors in the criticality tier of the application. The formula is configurable: organizations assign weights based on their risk appetite. SLA compliance follows the same pattern, comparing finding ages against severity specific thresholds to produce a compliance percentage.

Time series aggregation rolls up finding data at daily, weekly, and monthly intervals. Each roll up computes period metrics (new findings, resolved findings, net open count) and derived indicators (period over period change, moving averages, severity distribution shifts). These power the trend dashboards tracking organizational progress over time.

Threshold classification assigns categorical labels based on metric values. Risk tiers (critical, high, medium, low) are assigned to applications by composite score. Coverage gaps are flagged when a repository lacks scan records for a tool category. SLA breaches are marked when open finding age exceeds the severity specific threshold. These classifications enable filtering and alerting downstream.

Two refresh strategies apply. **Incremental refresh** recomputes only partitions affected by new silver records. This suits most aggregations where each record maps to a single grain partition (e.g., one application, one time period). **Full refresh** recomputes the entire table. It is necessary for metrics depending on global state, such as cross application percentile rankings or organization wide severity distributions.

2.5.1.3 ML Driven Analytics

ML driven analytics learn patterns from historical silver and gold data to produce predictive signals rule based formulas cannot capture. The framework supports four use cases: **composite risk scoring** learns which combinations of findings and development activity signals have preceded security incidents rather than using predefined weights, **false positive prediction** estimates the probability that a new finding is a false positive from historical triage decisions and repository history to reduce developer noise, **remediation time estimation** predicts MTTR from historical resolution data to inform SLA forecasting, and **anomaly detection** flags spikes in finding volumes or severity distribution shifts that reveal tool misconfigurations or new weak points.

Across all four, framework commitments are narrow. Outputs land in Gold tables under the Gold Aggregation pattern from Section 2.2.2, every run is tracked in MLflow with the

run ID recorded on output rows, and retraining ships in the same Databricks Asset Bundle (DAB) bundle as ingestion and transformation so deploys and rollbacks are atomic. Feature Store supplies reusable feature tables across training and inference, and Model Serving exposes real time inference endpoints cached in Lakebase for latency sensitive predictions (Databricks, 2025a; MLflow, 2025). Feature selection, splits, algorithms, and cadence are left to Chapter 3.

2.5.2 Serving Patterns

The serving layer delivers gold outputs to the data consumers from Section 2.1.2. Three delivery modes address different access patterns and latency requirements.

2.5.2.1 Analytical serving

Analytical serving targets dashboards, reporting tools, and ad hoc analysis through the OLAP path. A SQL warehouse executes queries against gold Delta tables, providing sub five second response times for precomputed aggregations. Materialized views cache frequently accessed query patterns, reducing compute cost for recurring dashboard refreshes.

Stakeholder specific views organize gold data for each audience: executive risk overviews show application risk scores and organizational trends, team dashboards present remediation metrics and SLA compliance, and security engineering views expose tool coverage gaps and deduplication statistics. These are SQL views on top of the gold tables, not separate copies, ensuring consistency across consumers.

2.5.2.2 Operational serving

Operational serving targets low latency workloads through the OLTP path. Lakebase exposes gold tables as PostgreSQL queryable relations with sub 50 millisecond response times (Databricks, 2026b). Because Lakebase shares storage with the lakehouse, gold tables are accessible without data duplication or synchronization pipelines.

Three operational workloads use this path. Remediation state lookups retrieve current status, severity, and ownership of individual findings for developer workflow integration. Operational APIs expose risk scores, coverage data, and finding details through the PostgREST Data API, providing HTTP access to any gold table without custom API code. ML inference results cached in Lakebase serve real time risk assessments for newly ingested findings. Lakebase scales to zero, reducing cost during off peak periods.

2.5.2.3 Event driven serving

Event driven serving pushes data to external systems rather than waiting for them to query. This closes the loop between analysis and operational action.

Automated issue creation triggers when new critical findings or SLA breaches are detected. The framework creates work items in issue trackers (Jira, ServiceNow) through their APIs, linking back to the finding record. Idempotency guards prevent duplicate ticket creation on reruns.

Threshold based notifications alert stakeholders when metrics cross configured boundaries: a risk score exceeds a threshold, a new KEV listed vulnerability is detected, or SLA compliance drops below a target percentage. Notifications are delivered through configured channels (email, messaging platforms, webhooks).

SIEM/SOAR event feeds push structured security events to platforms such as Lakewatch (Section 1.7.2). CDC on gold tables generates a stream of change events (new critical findings, risk score changes, coverage regressions) that SOAR playbooks consume for automated response. Shared infrastructure with Lakewatch (Section 2.1.1) enables this integration without external data movement.

Bidirectional synchronization keeps the view in the framework consistent with external systems. Issue tracker status updates (e.g., a Jira ticket moved to "Done") flow back through scheduled polling or webhooks, updating the lifecycle state in silver for the corresponding finding. This ensures gold layer remediation metrics reflect actual resolution progress.

2.5.3 Testing and Validation

Rule based analytics are validated by snapshot comparison. Each test supplies silver test cases with known values and confirms that the gold output matches expected table exactly, using PySpark DataFrame assertions on aggregation outputs. Test cases cover the computation boundaries from Section 2.5.1: zero findings, all critical findings, missing enrichment data, period bucketing at month boundaries, and threshold classification at each tier cutoff. A refresh idempotency case runs the pipeline twice on the same input to confirm no spurious gold writes.

ML driven analytics are validated by held out evaluation and drift monitoring. MLflow's `evaluate()` API computes accuracy, precision, recall, and F1 score on a held out dataset, and `validate_evaluation_results()` gates promotion against configured thresholds and the registered production baseline (MLflow, 2025). In production, prediction distribution comparisons against a rolling baseline flag drift before model quality visibly degrades. Feature Store staleness checks ensure inference features are no older than their configured threshold. Serving endpoints are exercised through HTTP clients to verify correctness and measure

latency against the targets in Section 2.5.2.

The required test suite covers:

- Every gold table: metric correctness, boundary conditions, and refresh idempotency.
- Every ML model: training convergence, validation accuracy, drift monitoring, and promotion gating.
- Every serving endpoint: response correctness and latency verification.

New analytics or serving configurations do not merge until this suite passes.

2.6 Extension Blueprint and AI Assistance

Each artifact specified in this chapter is a template rather than unrestricted, free-form code. This covers the connector files (Section 2.4), the analytics jobs (Section 2.5), and the test suites (Section 2.4.4, Section 2.5.3). This was an objective. The framework should be suited to AI assisted extension. The external requirements specification provides three Claude Code skills that consume these templates and produce a reference implementation. The skills are organized by phase: `analyze-source` maps a new data source onto the framework, `generate-connector` produces the eight connector artifacts listed in Section 2.4, and `validate-implementation` runs the test suite from Section 2.4.4 until the requirements from the specification are mapped to passing tests.

It was a conscious design decision that the set consists of three skills rather than a dedicated skill for every combination of role and application security category. This follows the recommended pattern for skills that span multiple variants of the same task (Anthropic, 2026). Each `SKILL.md` carries the role wide procedure. Guidance per category lives one level deeper, in a `references/<category>.md` file that the agent reads on demand. The seven application security categories are `cmdb`, `scm`, `sast`, `sca`, `secret`, `dast`, and `waf`.

This structure inherits two properties from the underlying skills mechanism (Agent Skills working group, 2026; Anthropic, 2025).The metadata overhead in the agent's system prompt remains fixed at three descriptions, regardless of how many categories are eventually added to the framework. The procedure is authored once and shared across categories. Chapter 3 reports on the execution of the catalog against the selected sources. Manually writing connectors is mundane. The approach of this thesis relies on modern AI assistance. A well specified framework plus a compact skill set makes the platform reproducibly constructible, with full traceability from test to requirement directly in the specification.

3. MVP Implementation

This chapter reports the reference implementation of the framework defined in Chapter 2. The deliverable is a Databricks Asset Bundle (Databricks, 2025e) that instantiates the three ingestion categories against the nine sources selected in Section 1.8. It is a minimum viable product, not a production ready integration of every security tool an enterprise might deploy. The nine sources cover the ingestion and integration patterns the framework must support: two SCM platforms, four scanners across static and dynamic testing, a CMDB, a secrets tool, and a runtime security source. Onboarding a tenth source does not require a framework change. It repeats the procedure in Section 3.1 and lands a new `src/connectors/<source>/` folder.

The contribution is twofold. First, the contract of three categories holds across the nine sources and the resulting code fits one small, repeating module layout. Second, the connectors were demonstrably generated end to end against the nine sources via a chain of three skills, with reviewer subagent cycles catching named defect classes (see Section 3.6.3 and Section 3.6.3). Section 3.2 reports the realized project layout. Section 3.3 resolves each source to its category. Section 3.4 shows the two ends of the category range in concrete code: ServiceNow on Lakeflow Connect (declarative) and GitHub on PyGitHub (PyGithub Contributors, 2025) (SDK).

3.1 Methodology

The implementation follows the same five steps for every source in Section 1.8:

1. **Resolve the ingestion category.** The source is matched against the three categories defined in Section 2.4.1. The evaluation order is fixed: Lakeflow Connect (Databricks, 2025c) first, then a maintained Python SDK, then dlt. Tools that only ship a CLI fall outside the axis of three categories and follow the artifact pattern at the CI/CD step described in Section 2.4.2. Section 3.3 carries out this step for all nine sources.
2. **Instantiate the module layout for the connector.** Every connector carries the same files under `src/connectors/<source>/`: `ingest.py`, `transform.py`, `mapping.yml`, `config.yml`, `severity.yml`, and `status.yml`. Bundle resources, tests, and the secret loading script are colocated with the connector code. For Lakeflow Connect sources, `ingest.py` is reduced to a module docstring and ingestion is declared in the bundle fragment as a `pipelines` resource with an `ingestion_definition`.
3. **Apply the mapping declaratively.** The `mapping.yml` file for the connector encodes the column expressions from bronze to silver given by the published mapping requirements. Severity and status lookups live in the connector folder as `severity.yml` and `status.yml`. `src/platform/silver.py` provides severity rank and status bucket normal-

ization helpers and deduplication utilities shared across connectors. The `transform.py` module builds the silver DataFrame field by field, calling the shared helpers. The current MVP builds the DataFrame in Python rather than applying the YAML declaratively. Section 3.6.3 carries the declarative applicator as a thread.

4. **Test in the layered strategy from Section 2.4.4.** Tests divide along the signature of the code they exercise. Row level logic in pure Python (parser logic, mapping application, High Water Mark (HWM) state, dedup key construction) runs locally against `list[dict]` fixtures with no `SparkSession`. DataFrame logic (transformations, envelope stamping, schema enforcement) runs through a session scoped Databricks Connect fixture that attaches to a remote workspace, so the test runtime matches the production runtime. Tests live under `src/connectors/<source>/tests/` and are anchored to requirement IDs through `@pytest.mark.requirement` markers.
5. **Run the skill chain pass for the source.** A chain of three Claude Code skills (analyze-source, generate-connector, validate-implementation) executes against the source and writes evidence rows to the operator docs site: a Generation log entry per pass and a Validation table keyed by requirement IDs. Each skill pass is then reviewed by a separate subagent (one for spec compliance, one for code quality), whose findings are folded back as follow up commits before the next source moves forward. Section 3.6.3 records the defect classes the reviewer cycle caught.

Step 1 has a deterministic answer for every source in the sample. Step 2 has an invariant layout across categories. That invariance is what makes the framework instantiable by an AI pipeline. A generator that emits the layout for a new source does not need to branch on category beyond what `ingest.py` contains and which bundle resource kind the source requires. Step 5 makes that generation procedure auditable: the operator docs site at `https://vkraus.github.io/appsec-mvp/` carries the per source Generation log and Validation table as the trail of evidence.

3.2 Project Structure

The reference implementation is distributed across two repositories that separate the thesis document from the MVP code.

- The thesis repository holds the LaTeX sources for this document.
- The `appsec-mvp` repository (`https://github.com/vkraus/appsec-mvp`) holds the reference implementation described in the remainder of this section. The operator docs site lives inside the same repository under `mkdocs/` and publishes at `https://vkraus.github.io/appsec-mvp/` via GitHub Pages. The published site is the operator facing reference and the trail of evidence for each connector.

3.2.1 Layering rule

The repository is organized into three install layers with no upward or sideways dependencies in setup code: platform, then connectors, then analytics. The platform layer must not pre-declare resources for any specific connector. No source schemas, volumes, connections, or secrets live in `src/platform/resources/`. Each connector owns its own bronze and silver schemas, secret loading script, and bundle resources. The analytics layer reads only the connector agnostic silver tables (`silver.findings`, `silver.repositories`, `silver.app_repo`, `silver.hwm`). The one ordering constraint is data level, not setup code level. An SCM connector (GitHub or GitLab) must run first at job time because non SCM connectors map findings onto repository entities populated by SCM. The bundle deploys all declared resources regardless of install order.

3.2.2 Component colocation

Inside the `appsec-mvp` repository, every component (the platform, each connector, the analytics layer) owns its code, its tests, its bundle resources, its scripts, and (for connectors that bring up source systems) an optional Terraform runtime. Adding a new source is one new folder under `src/connectors/`. There are no top level `tests/`, `config/`, `resources/`, `sql/`, or `infra/terraform/` directories. Source code 3.1 shows the realized layout with one connector of each category expanded.

Source code 3.1
Realized MVP project layout. One connector of each category is expanded; the others follow the same layout.

```
appsec-mvp/
|-- databricks.yml                          # bundle root: targets,
    variables
|-- pyproject.toml                          # Python package metadata
|-- mkdocs/                                 # operator docs site source
|-- examples/end-to-end-demo/               # demo across scanners
|-- src/
|   |-- platform/                           # framework primitives (runtime-
    shared)
|   |   |-- silver.py                       # severity/status normalization
    + dedup
|   |   |-- hwm.py, schemas.py, ...         # high-water-mark, StructTypes,
    helpers
|   |   |-- resources/{platform,bootstrap-job}.yml
|   |   |-- scripts/bootstrap.sh            # one time bootstrap after
    deploy
|   |   |-- sql/silver_tables.sql           # standard silver DDL
|   |   \-- tests/                          # unit tests for the primitives
|   |-- connectors/
|   |   |-- servicenow/                     # category: Lakeflow Connect
|   |   |   |-- ingest.py                   # docstring only; pipeline is
    declared
|   |   |   |-- transform.py
```

```
|   |   |   |-- mapping.yml, config.yml
|   |   |   |-- severity.yml, status.yml
|   |   |   |-- resources/{schemas,connection,pipeline}.yml
|   |   |   |-- scripts/load-secrets.sh
|   |   |   |-- runtime/                    # optional Terraform for source
    bring-up
|   |   |   \-- tests/
|   |   |-- github/                         # category: SDK (PyGitHub)
|   |   |   |-- ingest.py, transform.py
|   |   |   |-- mapping.yml, config.yml
|   |   |   |-- severity.yml, status.yml
|   |   |   |-- resources/{schemas,job}.yml
|   |   |   |-- scripts/load-secrets.sh
|   |   |   |-- runtime/
|   |   |   \-- tests/
|   |   \-- (gitlab, sonarqube, semgrep, owasp_zap, dependency_track,
|   |       trufflehog, aws_waf): same layout, varying depth
|   \-- analytics/                          # gold-layer SQL + jobs
|       |-- resources/{schemas,job}.yml
|       \-- sql/, tests/
```

The bundle resources for each source are colocated: `src/connectors/<source>/resources/` contains `schemas.yml` and either `job.yml` (for SDK and dlt connectors) or `pipeline.yml` plus `connection.yml` (for Lakeflow Connect connectors). The bundle root includes everything via the glob `src/{platform,connectors/*,analytics}/resources/*.yml`. Schemas, volumes, and secret loading scripts move with the connector folder, so installing or removing a connector does not touch shared files.

3.3 Ingestion Category Assignment

Each source is resolved to one of the three ingestion categories defined in Section 2.4.1 before its connector module is written. The evaluation order is fixed (Lakeflow Connect, then SDK, then dlt). Each entry below restates the functional requirements drawn from Section 1.8.4 and the source specific reference page, assesses the three categories against them, and records the selected category together with the library or mechanism that instantiates it. CLI only tools fall outside the axis of three categories and follow the CI/CD step artifact pattern described in Section 2.4.2. The outcomes feed directly into Section 3.5 and determine the bundle resource kind (`jobs` or `pipelines`) used for each source.

- **ServiceNow.** Category: Lakeflow Connect. Functional requirements (ServiceNow reference): periodic pull from the Table API and CMDB Instance API with offset pagination, `sys_updated_on` as the high water mark, OAuth 2.0 or basic authentication, no webhook. ServiceNow is a source supported natively by Lakeflow Connect, covering authentication, pagination, incremental state, and schema inference through platform configuration. Lakeflow Connect meets every requirement. The `ingest.py` for the connector is left empty. Ingestion is declared in the bundle fragment under `resources/`.
- **GitHub.** Category: SDK via PyGitHub. Functional requirements (GitHub reference): REST

plus GraphQL access to repositories, commits, pull requests, branch protection, code scanning alerts, secret scanning alerts, and Dependabot alerts. Cursor pagination on REST via `Link` headers and `pageInfo` cursors on GraphQL. PAT or GitHub App installation tokens. Primary rate limits of 5 000 requests per hour (PAT) or 15 000 for Enterprise GitHub Apps. Webhook preferred per Section 2.4.2. Lakeflow Connect does not support GitHub at MVP time. PyGitHub is the widely adopted community Python SDK. It covers every REST endpoint the framework needs, handles `Link` header pagination, and exposes rate limit status as first class attributes. The SDK category wins: `ingest.py` calls PyGitHub's `Repository`, `PullRequest`, and `CodeScanningAlert` accessors directly.

- **GitLab.** Category: SDK via python-gitlab (python-gitlab Contributors, 2025). Functional requirements (GitLab reference): REST access to projects, merge requests, pipelines, and vulnerability findings (Ultimate tier) or pipeline artifact parsing (non-Ultimate). Keyset pagination on REST, `pageInfo` on GraphQL. PAT or OAuth. Webhook coverage. No Lakeflow Connect support. `python-gitlab` is the project recognized Python client, covers the required endpoints, and handles keyset pagination transparently. Category selection parallels GitHub.
- **SonarQube** (SonarSource, 2024). Category: dlt. Functional requirements (SonarQube reference): REST Web API (`api/issues/search`, `api/hotspots/search`, `api/projects/search`). Offset pagination via `p` and `ps` parameters. `updateDate` high water mark. Token based authentication via HTTP Basic. JSON responses. No Lakeflow Connect integration. SonarSource does not publish and maintain an official Python SDK, and surveyed community clients are unmaintained at the Web API version the Long Term Support (LTS) server ships. dlt's `rest_api` source template covers all requirements declaratively. `ingest.py` composes a dlt pipeline.
- **Semgrep (Docker).** Category: artifact path, not an HTTP category. Functional requirements (Semgrep reference, CLI variant): produce a JSON report per commit from a CI/CD step. The report file is the authoritative interface. The source has no HTTP API in this deployment. The framework handles this through the CI/CD step pattern (Section 2.4.2): the CI/CD step runs `semgrep --json` and `databricks fs cp` to a Unity Catalog Volume. The `transform.py` for the connector reads the Volume directly. `ingest.py` is empty.
- **Semgrep (Cloud).** Category: dlt. Functional requirements (Semgrep reference, Cloud Platform variant): REST access to findings with cursor pagination, `since_date`/`updated_at` high water mark, bearer token authentication. No Lakeflow Connect integration, no official Python SDK. dlt's `rest_api` source covers the cursor pagination and incremental loading contract.
- **Dependency-Track** (OWASP Foundation, 2024b). Category: dlt. Functional requirements (Dependency-Track reference): REST access to projects, components, findings, and vulnerabilities. Offset pagination. `lastOccurrence` high water mark. API key header authentication. OpenAPI specification published by the project. No Lakeflow Connect integration. The project does not ship a first party Python SDK. Community SDK coverage is thin and partial for the finding retrieval endpoints the framework requires. The OpenAPI document informs the dlt source configuration directly.

- **TruffleHog** (Truffle Security, 2024). Category: artifact path, not an HTTP category. Functional requirements (TruffleHog reference): CLI tool with no server API. Emits line delimited JSON on stdout with per finding `Verified` status and commit SHA. Identical to Semgrep (Docker) in ingestion pattern. The CI/CD step redirects `trufflehog filesystem --json` to a file and uploads it to the Unity Catalog Volume. The `transform.py` for the connector reads and normalizes it.
- **OWASP ZAP.** Category: artifact path, not an HTTP category. Functional requirements (OWASP ZAP reference): orchestrate scans against target URLs and emit a JSON report per scan run. In the MVP, scans are executed in a CI/CD step that runs the official ZAP image against the target environment and uploads the resulting report to a Unity Catalog Volume via `databricks fs cp`. The `transform.py` for the connector reads the Volume directly. `ingest.py` is empty.
- **AWS WAF.** Category: SDK via boto3 (Amazon Web Services, 2025). Functional requirements (AWS WAF reference): `GetSampledRequests` for matched request samples over a time window. IAM signed API calls. ARN and tag resource identification. Event window high water mark. No Lakeflow Connect integration. `boto3` is the AWS maintained SDK covering every WAF API operation, handling IAM signature construction and pagination. `ingest.py` calls `boto3.client("wafv2").get_sampled_requests` directly.

Table 3.1 collects the nine assignments alongside the key functional requirement that drove each choice.

Across the nine sources, one resolves to Lakeflow Connect (ServiceNow), three to an SDK (GitHub, GitLab, AWS WAF), three to dlt (SonarQube, Semgrep Cloud, Dependency-Track), and three to the artifact path of Section 2.4.2 (Semgrep Docker, TruffleHog, OWASP ZAP). Semgrep contributes two instantiations (Docker CLI in CI/CD and the Cloud API), which is why the nine selected sources resolve to ten ingestion category rows.

3.4 Representative Connectors

This section shows the realized structure of a connector at each end of the category range. ServiceNow represents the Lakeflow Connect case, in which `ingest.py` is empty and the bundle fragment carries the ingestion specification. GitHub represents the SDK case, in which `ingest.py` composes client calls into iterators over bronze records.

3.4.1 ServiceNow: Lakeflow Connect Pipeline

The ServiceNow connector has no Python ingestion code. The `ingest.py` module carries only a docstring that points at the bundle fragment under `src/connectors/servicenow/resources/pipeline.yml`, reproduced in Source code 3.2. Authentication is carried by a Databricks Connection de-

Table 3.1
Ingestion category assignment across the nine selected sources, showing the category chosen for each source and the library or mechanism that instantiates it.

Source	Category	Binding	Decisive requirement
ServiceNow	Lakeflow Connect	platform managed	First party LFC supported source. Full contract met declaratively.
GitHub	SDK	PyGitHub	Mature Python SDK covers REST endpoints, `Link` header pagination, and rate limit state.
GitLab	SDK	python-gitlab	Project recognized Python client covers REST with keyset pagination.
SonarQube	dltool	`rest_api` source	No maintained Python SDK. REST only access with offset pagination and `updateDate` high water mark.
Semgrep (Docker)	Artifact path	CI to Volume	CLI tool with no HTTP API. Ingestion via CI/CD step pattern.
Semgrep (Cloud)	dltool	`rest_api` source	No Python SDK. Cursor paginated REST API with `since_date` high water mark.
Dependency-Track	dltool	OpenAPI driven	No first party SDK. REST with offset pagination and `lastOccurrence` high water mark.
TruffleHog	Artifact path	CI to Volume	CLI tool with no server API. Ingestion via CI/CD step pattern.
OWASP ZAP	Artifact path	CI to Volume	Scan report JSON produced in CI/CD and uploaded to Unity Catalog Volume.
AWS WAF	SDK	boto3	AWS maintained SDK covers `GetSampledRequests` and IAM signing.

clared in the same folder under `resources/connection.yml` and referenced by name. The `objects` block enumerates the two CMDB tables the silver layer consumes and their destination tables in bronze. Schedule and target catalog come from bundle variables so the same fragment can promote across dev, staging, and prod.

Source code 3.2

src/connectors/servicenow/resources/pipeline.yml

```
resources:
  pipelines:
    servicenow_ingest:
      name: servicenow_ingest
      catalog: ${var.catalog}
      target: bronze_servicenow
      continuous: false
      schedule:
        quartz_cron_expression: "0 0 * * * ?"
        timezone_id: UTC
      ingestion_definition:
        connection_name: ${var.servicenow_connection}
        objects:
          - table:
              source_schema: default
              source_table: cmdb_ci_business_app
              destination_catalog: ${var.catalog}
              destination_schema: bronze_servicenow
              destination_table: business_apps
          - table:
              source_schema: default
              source_table: cmdb_rel_ci
              destination_catalog: ${var.catalog}
              destination_schema: bronze_servicenow
              destination_table: app_cis
```

The framework prescribed connector layout is preserved. `mapping.yml`, `config.yml`, `sever ity.yml`, `status.yml`, and `transform.py` continue to govern the transformation from bronze to silver. Only `ingest.py` changes form, from a Python module to a declaration embedded in the bundle. That is the trade the Lakeflow Connect category offers. No pagination, authentication, or retry code written by hand, at the cost of binding the connector to a platform maintained capability.

3.4.2 GitHub: PyGitHub SDK Module

The GitHub connector uses PyGitHub. The `ingest.py` module exposes four names: a `github_client` factory and three iterator functions for organization repositories, repository commits, and repository pull requests. Source code 3.3 reproduces the iterator for repository commits as the representative case.

Source code 3.3

`src/connectors/github/ingest.py` *(one iterator shown)*

```python
from github import Auth, Github, GithubRetry


def github_client(token: str) -> Github:
    return Github(auth=Auth.Token(token), retry=GithubRetry(total=5))


def fetch_repo_commits(gh: Github, repo: str, since: str):
    since_dt = datetime.fromisoformat(since.replace("Z", "+00:00"))
    if since_dt.tzinfo is None:
        raise ValueError(f"`since` must be timezone-aware: {since!r}")
    for commit in gh.get_repo(repo).get_commits(since=since_dt):
        yield commit.raw_data
```

Three properties of this module are worth noting. First, authentication, pagination, and retry are delegated to the client library. `GithubRetry` is the `urllib3.Retry` subclass shipped by PyGitHub. It recognizes GitHub's secondary rate limit signal, which arrives as HTTP 403 with a `Retry-After` header, and which a plain `Retry(status_forcelist=[429, 5xx])` does not cover. Second, the boundary validation on `since` rejects timezone naive input explicitly rather than letting PyGitHub silently interpret the value. Third, the iterator yields the raw JSON body returned by the API via PyGitHub's `raw_data` attribute, preserving the schema on read contract for bronze.

The tests under `src/connectors/github/tests/` fake PyGitHub at the object graph level rather than at the HTTP level. A test fixture builds `MagicMock` instances modeled on PyGitHub's `Github`, `Organization`, `Repository`, and `PaginatedList` classes, each exposing the attributes the production code calls. The contract under test is the composition of SDK calls in the connector, not the HTTP behavior of the library.

3.5 Aggregate Results

Of the nine sources resolved in Section 3.3, all nine are delivered as production ready connectors under `src/connectors/<source>/`. Every connector carries the full set of files (`ingest.py`, `transform.py`, `mapping.yml`, `config.yml`, `severity.yml`, `status.yml`), bundle resources, secret loading scripts, and unit tests. Five connectors (ServiceNow, GitHub, SonarQube, Semgrep, OWASP ZAP) additionally ship an optional Terraform runtime under `runtime/` for source system bring up. Per source evidence is documented on the operator docs site.

The skill chain pass under step 5 of Section 3.1 ran end to end against all nine sources. Each source page on the operator docs site carries a Generation log with three rows (analyze, generate, validate), each row referencing the commit that produced it. Each source page also carries a Validation table keyed by requirement IDs that maps directly to `@pytest.mark.requirement` markers in the test suite. The cross source matrix at platform/reference/catalog aggregates the per source tables into one view. Both the per source pages and the catalog form the auditable trail of evidence behind the AI claim

defended in Section 3.6.3.

The aggregate test counts at HEAD are 184 passed, 15 skipped, 0 failed. Tests run under the layered strategy of step 4: pure Python tests run locally without a `SparkSession`, and DataFrame tests run through a session scoped Databricks Connect fixture against a remote workspace. The skipped tests are integration suites gated on live tenant credentials.

3.6 Discussion

3.6.1 Contract of three categories

Every source in the sample resolved to exactly one of Lakeflow Connect, SDK, or dlt, with the two CLI only tools falling into the artifact path carve out defined in Section 2.4.2. No source required a category the framework does not admit. Section 3.3 is the operational demonstration of this. The two implemented connectors in Section 3.4 show that the assignment is realizable in working code, and the nine module folders under `src/connectors/` confirm that the pattern repeats.

The Lakeflow Connect and SDK categories produce visibly different connector layouts. The ServiceNow connector has no Python ingestion file of substance. The GitHub connector has a small, flat `ingest.py` composed of SDK calls. Both preserve the module layout prescribed by the framework. The difference lives in `ingest.py` content and in which kind of DAB resource (`pipelines` or `jobs`) declares the scheduling. This is the trade the framework makes explicit. The Lakeflow Connect category absorbs ingestion into the platform at the cost of platform lockin. The SDK category keeps ingestion visible in Python at the cost of maintaining client knowledge for each source.

3.6.2 Declarative mapping

The connectors share severity and status normalization helpers in `src/platform/silver.py` and keep severity and status lookup tables in YAML next to the connector code (`severity.yml`, `status.yml`). The `transform.py` file for each source builds the silver DataFrame field by field and calls the shared helpers. Adding or remapping a field is currently a Python edit to the relevant `transform.py`. The `mapping.yml` file documents the intended derivation. Aligning the MVP with the declarative mapping prescription of Section 2.4.3 is a Section 3.6.3 thread.

3.6.3 Iteration Summary

This subsection collects implementation phase iterations that were caught by review rather than passing on the first attempt. Each is classified against the three way rubric formalized

in Section 3.6.3. **Framework gap** means extending the framework specification would have prevented the issue and would plausibly generalize. **Source quirk** means the correction is source specific noise not attributable to the framework. **Environment glue** means the issue concerns setup, authentication, or infrastructure outside the scope of the framework. Earlier iterations under the preredesign layout (a hand written shared HTTP primitive that introduced an unsanctioned fourth ingestion category, a missing boundary validator on incremental state inputs, a misconfigured retry policy on the GitHub client) are subsumed: the structural redesign closed both preredesign framework gaps by enforcing the layering rule of Section 3.2.1 and by anchoring the connector contract in the colocated test suite. The findings below are the postredesign reviewer cycle.

- **Layering boundary that the redesign closed.** The preredesign layout mingled top level `tests/`, `config/`, `resources/`, `sql/`, and `infra/terraform/` folders, and the platform layer predeclared resources for specific connectors. This permitted exactly the failure mode that the unsanctioned fourth ingestion category had exposed. The redesign moved every component to its own folder, deleted the predeclared resources from the platform layer, and stated the layering rule explicitly. **Framework gap.** The framework now states three install layers with no upward or sideways setup code dependencies, and the data level SCM first ordering is recorded separately as a job time constraint.
- **Schema drift between SQL DDL and PySpark types.** Splitting the `sql/` tree into per layer ownership exposed inconsistencies between the Data Definition Language (DDL) for the silver tables and the `StructType` definitions in `src/platform/schemas.py`. The reviewer flagged the drift and the spec recorded connector side population of `silver.repositories` and `silver.app_repo` as deferred work. **Framework gap.** The framework should hold a single source of truth for silver schemas. Section 3.6.3 carries the consolidation as a thread.
- **Secret scope mismatch in the GitHub ingest entry.** The post deploy bootstrap script defines a secret scope named `mvp-connectors`. The GitHub ingest entry point read from a scope named `appsec`, a residue from an earlier prototype. The reviewer caught the mismatch during the bootstrap script review. **Environment glue.** The fix aligned the entry point with the documented scope name.
- **Worktree pollution.** Several review cycles surfaced files that the redesign deleted reappearing on disk and being staged for inadvertent reintroduction (likely an editor or antivirus recreating them). Reviewers caught the pattern before bad commits landed. **Environment glue.** The pattern is not attributable to the framework.
- **Cron schedule collision between GitHub and SonarQube.** The GitHub connector job runs every fifteen minutes. The SonarQube connector was originally scheduled at `0 15 */3 * * ?`, which collides with the GitHub schedule at `00:15`, `03:15`, and so on. The reviewer flagged the collision and the schedule was changed to `0 17 */3 * * ?`. **Source quirk.** The correction is specific to two job schedules and does not generalize.
- **IRSA OIDC trust policy gaps.** Migrating each connector Terraform runtime under `src/connectors/<source>/runtime/` surfaced incomplete trust policy details on the IAM Roles for Service Accounts (IRSA) bindings. The reviewer required full trust policy

declarations and explicit operator inputs for OIDC issuer URLs. **Environment glue.** The corrections concern AWS account setup and are documented per connector under the `runtime/README.md` file.

Two of the six postredesign findings classify as framework gaps. Both name a specific consolidation the framework should hold (the layering rule, a single source of truth for silver schemas). Three classify as environment glue and one as a source quirk. The reviewer cycle caught all six before the commits landed, which is the empirical evidence behind the claim that the skill chain plus reviewer pattern catches named defect classes. Section 3.6.3 lifts these findings to a defended claim about where the framework holds and where it admits further specification.

Conclusion

Thesis Outcomes and Contributions

This thesis delivers the three contributions announced in the abstract, each targeted in a dedicated chapter and evaluated against the design science framework of Hevner et al. (2004) and the outcome evaluation guidance of Peffers et al. (2007). The first contribution is a **requirements specification** for an application security data platform. Chapter 1 motivates the specification, surveys the data sources and integration patterns that inform it, and catalogs the functional and nonfunctional requirements it must satisfy. Because of its substantial length, the specification text was moved to the product documentation website and included as a thesis attachment instead of being placed in an appendix, and Chapter 1 refers to the website where appropriate. The second contribution is a **reusable and extensible framework** delivered in Chapter 2, covering the reference architecture, the data model, and the medallion based pipeline (Strengholt, 2025) that carries source records through to the consumption layer. The third contribution is a **reference implementation** delivered in Chapter 3, built on the Databricks platform and instantiating the framework against selected data sources.

The reusability and extensibility claim of the framework is defended empirically by Chapter 3. The reference implementation instantiates the framework against selected sources with distinct integration patterns. For instance, ServiceNow is ingested through Lakeflow Connect as a managed connector, and GitHub is ingested through PyGitHub under the connector contract specified in Section 2.4. The framework is **reusable** in the concrete sense that Chapter 3 introduced no new connector level design decisions beyond the choice of which sources to onboard. Every architectural question had already been answered by the framework contract. The framework is **extensible** in the sense that onboarding a new source reduces to the four step procedure described in Section 3.1. That procedure covers ingestion category resolution, module instantiation per connector, declarative mapping, and layered testing, and it leaves no design work at the connector level.

Evaluation of AI Instantiability

Chapter 2 aimed the outputs of the framework at consumption by an AI coding agent. The original methodology target was fully autonomous generation of the reference implementation from the agent skill catalog on the documentation site. The MVP delivered in Chapter 3 was produced by invocations of that catalog under reviewer subagent supervision, following the Subagent-Driven Development pattern.

Acceptance criterion

A correction counts as a **framework gap** when two conditions are met:

- Extending the framework specification (schema pattern, connector contract, or declarative artifact such as `mapping.yml`) would have produced the right output on first pass.
- The extension would generalize to other sources in the same category. Corrections that trace to source specific noise (**source quirks**) or to setup and authentication plumbing outside scope (**environment glue**) do not count against the framework.

Empirical outcome

The skill chain ran end to end against all nine selected sources. It produced four reconciliation passes against connectors that already existed and five greenfield generations for sources that did not. Supervision was provided by reviewer subagent cycles per the Subagent-Driven Development pattern, with each pass reviewed for spec compliance and code quality before the next source moved forward. Section 3.6.3 enumerates the defect classes the reviewer cycle caught: a layering boundary that the redesign closed (framework gap), schema drift between SQL DDL and PySpark types (framework gap), a secret scope mismatch in the GitHub ingest entry (environment glue), worktree pollution across review rounds (environment glue), a cron schedule collision between GitHub and SonarQube (source quirk), and IRSA OIDC trust policy gaps in connector runtimes (environment glue). Two of six findings classify as framework gaps and name specific consolidations the framework should hold. The evidence for these findings lives in the implementation history report and in the per connector Generation logs published on the documentation site at `https://vkraus.github.io/appsec-mvp/`.

Claim defended

The framework is AI **instantiable** under skill catalog invocation. The reference implementation was demonstrably generated end to end with reviewer subagent cycles against nine sources. The framework contract, the mapping, and the module layout per connector together reduce new connector work to a four step procedure (Section 3.1) that converges with bounded reviewer supervision. The framework is not yet AI **autonomous**. The empirical build still required reviewer subagent cycles on every connector, and the framework gaps named in Section 3.6.3 have not been closed. Closing those gaps and refining the skill prompts and reviewer rubrics until invocations are self sustaining would measure the distance from instantiable to autonomous. That measurement is the subject of Future Work.

Limitations

All nine sources selected in Section 1.8 are delivered as production ready connectors with the full file set, bundle resources, secret loading scripts, and unit tests. The reference implementation is **platform specific**. It runs on Databricks over AWS object storage, and that is the only combination of runtime and cloud exercised end to end. The connector contract in Section 2.4 is written to be cloud agnostic and platform agnostic in principle, yet it has not been ported to Snowflake, to Microsoft Fabric, or to an on premises platform. The portability claim of the framework therefore rests on the design of the contract, not on a second implementation.

The **reproducibility** of the reference implementation rests on preconditions enumerated on the product documentation site rather than in the thesis itself. An independent replication requires a Databricks workspace with Unity Catalog enabled, an AWS account with an S3 bucket and object storage credentials federated into the workspace, a Lakeflow Connect entitlement on that workspace, a ServiceNow developer instance with an API capable user, and a GitHub Personal Access Token scoped to the organization under test. The infrastructure is bootstrapped by a Terraform module carried in `mvp/terraform/` and the pipeline is deployed by a Databricks Asset Bundle. Both expect a target prefix and credentials as input variables. A reader who retains only the submitted `mvp.zip` and `docs.zip` cannot reproduce the build without the workspace entitlements listed above.

Several **open implementation items** remain. Population of `silver.repositories` and `silver.app_repo` from the SCM connectors is pending: the DDL exists at the platform layer, but the writers per connector have not been wired up, so joining scanner findings to repositories is structurally supported and produces empty results. Default tag propagation that the original Terraform tree carried on the AWS provider is lost in the per connector runtimes, so operators relying on cost allocation tags lose them. The analytics layer is empty: `src/analytics/` carries placeholder schemas and a placeholder job pointing at a `.placeholder` SQL file.

The **instrument measurement conflict** in Section 3.6.3 is real and not mitigated by the acceptance rubric. The framework specification, the skill catalog, the delivered reference implementation, and the iteration log that grades the fitness of the framework are all authored by the same operator in the same session, so the framework was revised in response to iteration outcomes that the evaluation then treats as measurements of fitness. Under this loop, a verdict that "framework gaps were caught" is unfalsifiable. A blind operator applying a frozen version N of the specification to source 10 would produce a stronger measurement. That measurement is the controlled user study queued in Section 3.6.3.

The **variation** of the selected sources is biased toward REST JSON APIs with token authentication. Seven of the nine sources (ServiceNow, GitHub, GitLab, SonarQube, Dependency-Track, OWASP ZAP, AWS WAF) fall into this category, which is also the category Lakeflow Connect and maintained Python SDKs support. The two artifact file sources (Semgrep CLI, TruffleHog)

deliver reports via S3 rather than an interactive API. Source categories against which the framework has **not** been demonstrated include SOAP APIs, gRPC, syslog streams, Kafka event streams, binary protocol telemetry, and push only webhook native sources. The contract in Section 2.4 is written to accommodate these categories, but evidence supporting that claim is absent on a category by category basis.

Future Work

The most immediate area of focus is **closing the framework gaps** identified in Section 3.6.3. The two framework gaps name specific consolidations the framework should hold: an enforced layering boundary already addressed by the redesign, and a single source of truth for silver schemas that resolves the drift between SQL DDL and PySpark `StructType` definitions. Closing the schema drift gap is the precondition for any stronger AI autonomy claim.

A closely related thread is **completing the declarative mapping applicator**. The transformation patterns prescription in Section 2.4.3 treats the `mapping.yml` in each connector as the authoritative specification from bronze to silver consumed by a generic applicator, but the MVP implements the transformation field by field in the `transform.py` of each connector (Section 3.6). Completing the applicator so that `mapping.yml` drives the silver DataFrame construction would close the gap between the framework contract and the realized code, and would make the addition cost per source a configuration edit rather than a code edit.

The third thread is **populating `silver.repositories` and `silver.app_repo` from the SCM connectors**. The DDL is in place at the platform layer, and the layering rule reserves the SCM first ordering at job time, but the writers per SCM and CMDB connector have not been implemented. Until those land, the gold layer cannot join scanner findings to repository entities for the analytics that Section 2.5.1 prescribes.

The fourth thread is **refining the published skill catalog by defect class**. The structural redesign of the catalog is no longer in scope: the form is settled per the Note on Extension and AI Assistance at the close of Chapter 2, which uses three role wide skills with specialization per category in `references/<category>.md`, defended against the recommended pattern for skills that span multiple variants of the same task. What remains is per defect class refinement of the skill prompts and the reviewer rubrics. Tightening the preconditions asserted in each `SKILL.md` against the framework gap and source quirk classes named in Section 3.6.3, and widening the reviewer rubrics to catch the environment glue classes earlier, would reduce the supervision cost per invocation and would move the published catalog toward self sustaining operation.

The fifth thread is **platform portability**. The reference implementation runs only on Databricks over AWS. A second implementation on Snowflake or Microsoft Fabric would validate the portability claim of the connector contract empirically rather than on contract

design alone, and would expose any residual coupling to runtime assumptions specific to Databricks.

The sixth thread is **downstream analytics on the gold layer**. The gold tables are positioned for consumption, but no consumer has been implemented against them. Large Language Model assisted triage and correlation over the gold layer is a natural extension and would exercise the data model in the direction it was designed for.

The seventh thread is a **controlled user study**. The AI feasibility evaluation is a demonstration by a single operator. A study across multiple operators, or across sources handed to operators blind, would produce statistical evidence rather than existence proof evidence.

References

Agent Skills working group. (2026). *Agent skills specification*. Retrieved April 25, 2026, from https://agentskills.io/specification (cit. on p. 68).

Amazon Web Services. (2025). *boto3: AWS SDK for Python*. Retrieved April 24, 2026, from https://boto3.amazonaws.com/v1/documentation/api/latest/index.html (cit. on p. 74).

Amazon Web Services. (2026). *AWS WAF developer guide: Getting information about requests with GetSampledRequests*. Retrieved April 19, 2026, from https://docs.aws.amazon.com/waf/latest/APIReference/API_GetSampledRequests.html (cit. on pp. 14, 34).

Anderson, R. (2020). *Security engineering: A guide to building dependable distributed systems* (3rd ed.). John Wiley & Sons. (Cit. on pp. 12, 22, 23).

Anthropic. (2025). *Equipping agents for the real world with agent skills*. Retrieved April 25, 2026, from https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills (cit. on p. 68).

Anthropic. (2026). *Skill authoring best practices*. Retrieved April 25, 2026, from https://platform.claude.com/docs/en/agents-and-tools/agent-skills/best-practices (cit. on p. 68).

Armbrust, M., Das, T., Sun, L., Yavuz, B., Zhu, S., Murthy, M., Torres, J., van Hovell, H., Ionescu, A., Łuszczak, A., Świtakowski, M., Szafrański, M., Li, X., Ueshin, T., Mokhtar, M., Boncz, P., Ghodsi, A., Paranjpye, S., Senster, P., … Zaharia, M. (2020). Delta Lake: High-Performance ACID table storage over cloud object stores. *Proceedings of the VLDB Endowment*, *13*(12), 3411–3424. https://doi.org/10.14778/3415478.3415560 (cit. on p. 36).

Armbrust, M., Ghodsi, A., Xin, R., & Zaharia, M. (2021). Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics. *Proceedings of the 11th Annual Conference on Innovative Data Systems Research (CIDR)* (cit. on pp. 27, 36).

AXELOS. (2019). *ITIL foundation: ITIL 4 edition*. TSO (The Stationery Office). (Cit. on pp. 12, 17–19).

Chacon, S., & Straub, B. (2014). *Pro git* (2nd ed.). Apress. (Cit. on p. 20).

Chess, B., & West, J. (2007). *Secure programming with static analysis*. Addison-Wesley. (Cit. on pp. 12, 23).

Cybersecurity and Infrastructure Security Agency. (2024). *Known exploited vulnerabilities catalog*. Retrieved April 11, 2026, from https://www.cisa.gov/known-exploited-vulnerabilities-catalog (cit. on p. 22).

DAMA International. (2017). *DAMA-DMBOK: Data management body of knowledge* (2nd ed.). Technics Publications. (Cit. on pp. 27, 28, 60).

Databricks. (2025a). *Databricks Feature Engineering*. Retrieved April 11, 2026, from https://docs.databricks.com/aws/en/machine-learning/feature-store/ (cit. on p. 66).

Databricks. (2025b). *Databricks Terraform provider*. Retrieved April 19, 2026, from https://registry.terraform.io/providers/databricks/databricks/latest/docs (cit. on p. 51).

Databricks. (2025c). *Lakeflow Connect*. Retrieved April 24, 2026, from https://docs.databricks.com/aws/en/ingestion/lakeflow-connect/ (cit. on p. 69).

Databricks. (2025d). *Lakeflow Spark Declarative Pipelines*. Retrieved April 11, 2026, from https://docs.databricks.com/aws/en/ldp/ (cit. on pp. 14, 33, 37).

Databricks. (2025e). *What are Databricks Asset Bundles?* Retrieved April 11, 2026, from https://docs.databricks.com/aws/en/dev-tools/bundles (cit. on pp. 51, 69).

Databricks. (2025f). *What is Unity Catalog?* Retrieved April 11, 2026, from https://docs.databricks.com/aws/en/data-governance/unity-catalog/ (cit. on pp. 14, 37).

Databricks. (2026a). *Databricks enters security market with launch of Lakewatch: New open, agentic SIEM*. Retrieved April 11, 2026, from https://www.databricks.com/company/newsroom/press-releases/databricks-enters-security-market-launch-lakewatch-new-open-agentic (cit. on pp. 13, 31).

Databricks. (2026b). *Databricks Lakebase is now generally available*. Retrieved April 11, 2026, from https://www.databricks.com/blog/databricks-lakebase-generally-available (cit. on pp. 37, 66).

Databricks. (2026c). *Lakeflow Jobs*. Retrieved April 11, 2026, from https://docs.databricks.com/aws/en/jobs/ (cit. on p. 53).

Databricks. (2026d). *Observability in Databricks for jobs, Lakeflow Spark Declarative Pipelines, and Lakeflow Connect*. Retrieved April 11, 2026, from https://docs.databricks.com/aws/en/data-engineering/observability-best-practices (cit. on p. 53).

DefectDojo. (2024). *DefectDojo documentation*. Retrieved March 6, 2026, from https://documentation.defectdojo.com/ (cit. on pp. 13, 30).

Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The science of lean software and DevOps*. IT Revolution Press. (Cit. on p. 21).

Forum of Incident Response and Security Teams. (2019). *Common vulnerability scoring system v3.1: Specification document*. Retrieved March 6, 2026, from https://www.first.org/cvss/v3.1/specification-document (cit. on p. 22).

Gardner, D., Zumerle, D., & Bhat, M. (2023). *Innovation insight for application security posture management*. Gartner. Retrieved April 11, 2026, from https://www.gartner.com/en/documents/4326999 (cit. on pp. 12, 13, 30).

Gartner. (2024). Market Share: IT Service Management, Worldwide, 2024 [Gartner subscriber content, accessed via institutional subscription. Figure cited from the ServiceNow summary at https://www.servicenow.com/blogs/2025/no-1-6-tech-workflow-segments]. (Cit. on p. 18).

GitHub. (2024a). *GitHub REST API documentation*. Retrieved April 11, 2026, from https://docs.github.com/en/rest (cit. on pp. 20, 24).

GitHub. (2024b). *Rate limits for the REST API*. Retrieved April 18, 2026, from https://docs.github.com/en/rest/using-the-rest-api/rate-limits-for-the-rest-api (cit. on p. 21).

GitHub. (2025a). *About GitHub Advanced Security*. Retrieved April 11, 2026, from https://docs.github.com/en/get-started/learning-about-github/about-github-advanced-security (cit. on pp. 12, 30).

GitHub. (2025b). *Octoverse 2025: The state of open source*. Retrieved April 11, 2026, from https://octoverse.github.com/ (cit. on p. 20).

GitLab. (2025). *Application security testing*. Retrieved April 11, 2026, from https://docs.gitlab.com/user/application_security/ (cit. on p. 30).

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, *28*(1), 75–105. https://doi.org/10.2307/25148625 (cit. on pp. 15, 81).

Howard, M., & LeBlanc, D. (2006). *Writing secure code* (2nd ed.). Microsoft Press. (Cit. on p. 12).

Inmon, W. H. (2005). *Building the data warehouse* (4th ed.). John Wiley & Sons. (Cit. on p. 27).

Jacobs, J., Romanosky, S., Adjerid, I., & Baker, W. (2020). Improving vulnerability remediation through better exploit prediction. *Journal of Cybersecurity*, *6*(1), tyaa015. https://doi.org/10.1093/cybsec/tyaa015 (cit. on pp. 12, 22).

JetBrains. (2025). *The state of CI/CD in 2025*. Retrieved April 11, 2026, from https://blog.jetbrains.com/teamcity/2025/10/the-state-of-cicd/ (cit. on p. 21).

Kim, G., Humble, J., Debois, P., Willis, J., & Forsgren, N. (2021). *The DevOps handbook: How to create world-class agility, reliability, & security in technology organizations* (2nd ed.). IT Revolution Press. (Cit. on pp. 21, 22).

Kimball, R., & Ross, M. (2013). *The data warehouse toolkit: The definitive guide to dimensional modeling* (3rd ed.). John Wiley & Sons. (Cit. on pp. 27, 29).

Lee, D., Wentling, T., Haines, S., & Babu, P. (2024). *Delta Lake: The definitive guide: Modern data lakehouse architectures with data lakes*. O'Reilly Media. (Cit. on pp. 14, 27, 28, 36).

Linux Foundation. (2024). *SPDX specification*. Retrieved March 6, 2026, from https://spdx.dev/specifications/ (cit. on pp. 22, 24).

Mack, S. D. (2023). *The DevSecOps playbook: Deliver continuous security at speed*. John Wiley & Sons. (Cit. on p. 22).

McGraw, G. (2006). *Software security: Building security in*. Addison-Wesley. (Cit. on pp. 12, 22).

Miloslavskaya, N., & Tolstoy, A. (2016). Big data, fast data and data lake concepts. *Procedia Computer Science*, *88*, 300–305 (cit. on p. 27).

MITRE Corporation. (2024a). *CVE program*. Retrieved March 6, 2026, from https://www.cve.org/ (cit. on p. 22).

MITRE Corporation. (2024b). *CWE – common weakness enumeration*. Retrieved April 11, 2026, from https://cwe.mitre.org/ (cit. on pp. 22, 25).

MLflow. (2025). *MLflow documentation*. Retrieved April 11, 2026, from https://mlflow.org/docs/latest/ (cit. on pp. 66, 67).

National Institute of Standards and Technology. (2004). *Standards for security categorization of federal information and information systems* (tech. rep. No. FIPS PUB 199). U.S. Department of Commerce. Retrieved April 18, 2026, from https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.199.pdf (cit. on p. 17).

National Telecommunications and Information Administration. (2021). *The minimum elements for a software bill of materials (SBOM)*. Retrieved March 6, 2026, from https://www.ntia.gov/sites/default/files/publications/sbom_minimum_elements_report_0.pdf (cit. on p. 24).

OASIS Open. (2024). *Static analysis results interchange format (SARIF) version 2.1.0*. Retrieved March 6, 2026, from https://docs.oasis-open.org/sarif/sarif/v2.1.0/sarif-v2.1.0.html (cit. on p. 22).

OCSF Project. (2024). *Open cybersecurity schema framework*. Retrieved March 6, 2026, from https://schema.ocsf.io/ (cit. on pp. 22, 30).

Ohm, M., Plate, H., Sykosch, A., & Meier, M. (2020). Backstabber's knife collection: A review of open source software supply chain attacks. *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, 23–43. https://doi.org/10.1007/978-3-030-52683-2_2 (cit. on pp. 12, 23).

OpenSSF. (2024). *Supply-chain levels for software artifacts (SLSA)*. Retrieved March 6, 2026, from https://slsa.dev/ (cit. on pp. 12, 24).

OWASP Foundation. (2021). *OWASP top 10:2021*. Retrieved March 6, 2026, from https://owasp.org/Top10/ (cit. on p. 22).

OWASP Foundation. (2024a). *CycloneDX bill of materials standard*. Retrieved March 6, 2026, from https://cyclonedx.org/ (cit. on pp. 22, 24).

OWASP Foundation. (2024b). *OWASP Dependency-Track documentation*. Retrieved April 11, 2026, from https://docs.dependencytrack.org/ (cit. on p. 73).

OWASP Foundation. (2024c). *OWASP ModSecurity core rule set*. Retrieved April 18, 2026, from https://coreruleset.org/ (cit. on p. 26).

OWASP Foundation. (2024d). *OWASP ZAP documentation*. Retrieved April 11, 2026, from https://www.zaproxy.org/docs/ (cit. on pp. 14, 33).

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3), 45–77. https://doi.org/10.2753/MIS0742-1222240302 (cit. on pp. 15, 81).

Pressman, R. S., & Maxim, B. R. (2019). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill. (Cit. on p. 20).

PyGithub Contributors. (2025). *PyGithub documentation*. Retrieved April 24, 2026, from https://pygithub.readthedocs.io/en/stable/ (cit. on p. 69).

python-gitlab Contributors. (2025). *python-gitlab: A Python wrapper for the GitLab API*. Retrieved April 24, 2026, from https://python-gitlab.readthedocs.io/en/stable/ (cit. on p. 73).

Reis, J., & Housley, M. (2022). *Fundamentals of data engineering: Plan and build robust data systems*. O'Reilly Media. (Cit. on pp. 27, 28, 60).

Semgrep. (2024). *Semgrep documentation*. Retrieved April 11, 2026, from https://semgrep.dev/docs/ (cit. on p. 12).

ServiceNow. (2024a). *CMDB instance API reference*. Retrieved April 18, 2026, from https://www.servicenow.com/docs/bundle/zurich-api-reference/page/integrate/inbound-rest/concept/cmdb-instance-api.html (cit. on p. 18).

ServiceNow. (2024b). *REST API authentication*. Retrieved April 18, 2026, from https://www.servicenow.com/docs/bundle/xanadu-api-reference/page/integrate/inbound-rest/concept/c_RESTAPI.html (cit. on p. 19).

ServiceNow. (2024c). *Scripted REST APIs*. Retrieved April 18, 2026, from https://www.servicenow.com/docs/bundle/zurich-api-reference/page/integrate/custom-web-services/concept/c_CustomWebServices.html (cit. on p. 19).

ServiceNow. (2024d). *Table API reference*. Retrieved April 18, 2026, from https://www.servicenow.com/docs/bundle/zurich-api-reference/page/integrate/inbound-rest/concept/c_TableAPI.html (cit. on pp. 18, 19).

Skoulikari, A. (2024). *Learning git: A hands-on and visual guide to the basics of git*. O'Reilly Media. (Cit. on p. 20).

Snyk. (2024). *Snyk API documentation*. Retrieved April 11, 2026, from https://docs.snyk.io/snyk-api (cit. on p. 12).

Sommerville, I. (2015). *Software engineering* (10th ed.). Pearson. (Cit. on p. 20).

SonarSource. (2024). *SonarQube web API documentation*. Retrieved April 11, 2026, from https://docs.sonarsource.com/sonarqube-server/latest/extension-guide/web-api/ (cit. on p. 73).

Stack Overflow. (2025). *2025 stack overflow developer survey*. Retrieved April 11, 2026, from https://survey.stackoverflow.co/2025/ (cit. on pp. 20, 21).

Stallings, W., & Brown, L. (2024). *Computer security: Principles and practice* (5th ed.). Pearson. (Cit. on pp. 12, 17, 22).

Strengholt, P. (2025). *Building medallion architectures: Designing with Delta Lake and Spark*. O'Reilly Media. (Cit. on pp. 14, 27, 33, 81).

Stuttard, D., & Pinto, M. (2011). *The web application hacker's handbook: Finding and exploiting security flaws* (2nd ed.). John Wiley & Sons. (Cit. on pp. 12, 25).

Thalpati, G. A. (2024). *Practical lakehouse architecture: Designing and implementing modern data platforms at scale*. Apress. (Cit. on p. 27).

Truffle Security. (2024). *TruffleHog documentation*. Retrieved April 11, 2026, from https://trufflesecurity.com/trufflehog (cit. on p. 74).

Williams, R., & Gonzalez, K. (2021). *3 steps to improve IT service view CMDB data quality*. Retrieved April 11, 2026, from https://www.gartner.com/en/documents/3994127 (cit. on pp. 12, 19).

Winters, T., Manshreck, T., & Wright, H. (2020). *Software engineering at Google: Lessons learned from programming over time*. O'Reilly Media. (Cit. on p. 20).

Appendices

A. Generative AI Use Disclosure

This appendix expands the Declaration on the Use of Artificial Intelligence from the front matter of this thesis. The five sections below disclose each use in the same order. The central use is the AI assisted direct coding of the reference implementation (Chapter 3), which is evaluated in Section 3.6.3 rather than disclosed here as incidental assistance.

The grammar and language style assistance described in the first section below used the Overleaf AI assistant on the thesis manuscript. All other AI assistance reported in this appendix used Claude models (Anthropic) accessed through the Claude Code CLI, with model versions spanning Claude Opus 4.5 through Claude Opus 4.7 over the drafting period of the thesis.

A.1 Text Editing

AI assistance feature in Overleaf online editor was used for grammar checking, style refinement, synonym replacement, sentence rephrasing, and other editing of the content I originally authored. I relied on the assistant throughout my drafts to identify errors, tighten phrasing, and highlight inconsistencies. I accepted, rejected, or revised the suggestions. **I did not rely on any AI assistant to autonomously generate thesis content. I wrote all new text material in the Overleaf editor, and only afterward revised it with AI support.**

Several review cycles were carried out across the entire thesis using the Claude Code tool with Claude Opus models to condense it to its target length. These iterations rephrased longer passages into more concise wording and relocated some details into the attached product documentation, which contains the requirements specification and reference material per data source. The restructuring decisions were mine. The AI assistant proposed cuts and rephrasing that I accepted, rejected, or revised.

The research question, chapter structure, major contributions, and central claims are all mine. Positioning against prior work (Section 1.7) and the conceptual domain model (Section 1.6.3) were also authored by me. Literature selection is mine, Claude Code was used to generate most bibliography entries in LaTex format.

A.2 Images

AI assistant generated TikZ figures from my sketch or textual description of the content and layout, and together we iterated them for correctness and readability.

A.3 Product Documentation (`docs.zip`)

The product documentation, attached as `docs.zip`, is AI assisted content.

I authored the three skills on the skills page in Claude Code tool, using Anthropic skill writing skill. Each skill carries one `SKILL.md` for the role and one `references/<category>.md` per application security category, following the published Anthropic Pattern 2 layout defended in the Extension Blueprint section of Chapter 2. I iterated each skill until it produced the intended output. The catalog was then used to generate connector code for the reference implementation, as disclosed in the Source Code section below..

A.4 Source Code (`mvp.zip`)

The reference implementation in Chapter 3 (submitted as `mvp.zip`) was built in two modes, both with AI assistance from Claude Code. The first mode was direct coding against the framework contract (Chapter 2) and the external requirements specification, using Claude Code conversationally through a brainstorm, plan, and execute loop. The first four connectors (ServiceNow, GitHub, Semgrep, OWASP ZAP) were initially produced this way.

The second mode was the skill chain described in the Product Documentation section above. It ran end to end against all nine selected data sources. Each pass wrote evidence rows to the product documentation (a Generation log entry and a Validation table keyed by requirement IDs).

Both modes followed the Subagent-Driven Development pattern from the Superpowers workflow (brainstorm, spec, plan, execute). Each task was implemented by a fresh Opus subagent and then reviewed by a separate specification compliance subagent and a separate code quality subagent. The reviewer cycles caught the defect classes recorded in Section 3.6.3. This is my main AI use. Section 3.6.3 evaluates it. The three skill files published on the documentation site (`analyze-source`, `generate-connector`, `validate-implementation`) are considered an AI executable part of the requirements specification.

A.5 Supporting Tools

I used AI assistance in Claude Code for maintaining LaTeX script files that were used to build this document.